FernUni.ch
UniDistance.ch

Faculty of Mathematics and Computer Science

Bachelor Thesis

Introduction to Formal Mathematics in Lean with an example in Topology

Submitted by:
Daniele Bolla
Student ID: 21-694-187

Supervisor:
Prof. David Loeffler

Submission date:
December 15, 2025

Declaration

I hereby declare that this thesis is my own work, and that no other sources have been used except those indicated in the bibliography. I confirm that generative AI tools were not used in developing any part of this thesis, though they may have been used for preliminary research purposes and flow refinement.

Place, Date **Signature**

Omegna (Italy), 9 December 2025

Abstract

This thesis serves as an introduction to formal mathematics using the Lean proof assistant. After a brief overview of the Curry–Howard correspondence, we explore how mathematical structures and properties are defined in Lean. Finally, we present a formalization of the topologist's sine curve, which has been merged into the Mathlib library as a modest contribution to the broader Lean community.

During the work on this thesis, I have explored new concepts in both mathematics and computer science, starting from logic and type theory within the Curry-Howard correspondence. The practice of formal and constructive mathematics has influenced my approach to mathematical reasoning. In building the rational numbers, I explored their algebraic construction using quotients, which is reflected in Lean through quotient types. I further examined how algebraic structures such as groups and rings are defined using structures and type classes. For more technical depth, I studied filters to generalize the notion of convergence in topology. The formalization of the topologist's sine curve has deepened my understanding of connectedness, closures, and continuity.

The full code accompany this project is available [here].

Acknowledgments

I would like to express my sincere gratitude to Prof. David Loeffler for the consistent support. His mentorship has been particualary beneficial in enlightening my path through formal mathematics, shaping the thesis and giving me a chance to actually contribute to Lean community.

Moreover, I deeply thank FernUni for the opportunity to pursue my studies. I feel honored to have been able to study mathematics at this stage of my life, through a flexible and high-quality program.

I hope to continue my studies in the future.

Contents

1 Introduction **11**

1.1 First Steps in Lean . . . 13

2 Logic and Propositions as Types **17**

2.1 Propositional Logic and Natural Deduction . . . 17

2.2 Primitive Types . . . 19

2.3 The Curry–Howard Correspondence . . . 22

2.4 Predicate Logic and Dependent Types . . . 24

2.5 Constructive and Classical Reasoning . . . 26

3 Structures, Type Classes, and Coercions **29**

3.1 Exploring Mathlib (The `Rat` Structure) . . . 31

3.2 Coercions and Type Casting . . . 35

4 Quotients and Algebraic Hierarchy **37**

4.1 Quotients . . . 37

4.2 Type Classes and Algebraic Hierarchy . . . 44

5 Topology in Lean **49**

5.1 Real Numbers and Topological Spaces . . . 49

5.1.1 The `TopologicalSpace` type class . . . 50

5.2 Filters . . . 51

5.2.1 Key filter concepts . . . 52

5.2.2 The `Tendsto` predicate . . . 52

5.2.3 From global to local continuity . . . 53

5.3 Connectedness and Path-Connectedness . . . 54

5.3.1 Mathlib's formalization . . . 55

5.3.2 Useful theorems about connectedness . . . 56

6 Formalizing the Topologist's Sine Curve **57**

6.1 Motivation . . . 57

6.2 Setup . . . 58

6.3 T is Connected 59
6.3.1 Refactoring: from tactics to terms 61
6.3.2 From S to T 62
6.4 T is not Path-Connected 65
6.4.1 Defining t_0 and the continuity argument 66
6.4.2 Finding t_1 and the positive x-coordinate 67
6.4.3 The interval argument 68
6.4.4 Constructing the oscillating sequence 69
6.4.5 The final contradiction 70
6.5 Conclusion 71
6.5.1 Reflections on the formalization 72
6.5.2 Future directions 72

Chapter 1

Introduction

In this chapter, we introduce the Lean proof assistant and explain why its type system makes it suitable for formalizing mathematics. We begin by motivating the relevance of expressive type systems, then present the basic syntax of Lean and preview the key idea of the Curry–Howard correspondence (Section 1.1). A detailed treatment of the Curry–Howard correspondence and its logical foundations follows in Chapter 2.

Lean is both a **functional programming language** and a **theorem prover**. We focus primarily on its role as a theorem prover. But what does this mean, and how can it be achieved?

A programming language defines a set of rules, semantics, and syntax for writing programs. To achieve a goal, a programmer must write a program that meets given specifications. Nordström, Petersson, and Smith [NPS90] (Section 1.1) distinguish two approaches to ensuring program correctness: **program verification** and **program derivation**. In **program verification**, the programmer first writes a program and then proves it meets the specifications. In **program derivation**, the programmer constructs a proof that a program with certain properties exists, then extracts a program from that proof.

A related, though distinct, concept is the difference between **static** and **dynamic type systems**. In a static type system, certain classes of errors are detected at compile time through type annotations; in a dynamic type system, such checks are deferred to runtime. Most mainstream programming languages occupy a middle ground, providing basic types for compile-time checking while leaving more complex specifications to be enforced at runtime.

Example 1.0.1 *In a dynamically typed language like JavaScript, variables can change type after they are created. For example, a variable defined as a number can later be reassigned to a string:*

```
let price = 100; // JavaScript recognizes this variable as a number
price = "100";   // "100" transforms the number 100 to a string
```

TypeScript is a statically typed superset of JavaScript. Unlike JavaScript, it performs type checking at compile time. This means we can prevent the previous behavior, while writing our code, simply by adding a type annotation:

```
let price: number = 100;
price = "100";  // Error: Type 'string' is not assignable to type
'number'
```

Now assigning a string to a variable with type annotation `number` *results in a compile-time error.*

The point of this comparison is not that Lean is "better" than TypeScript; they serve different purposes. TypeScript's type system is designed for pragmatic software engineering, whereas Lean's is designed to express arbitrary mathematical propositions. As with most programming languages, TypeScript can only enforce complex specifications at runtime. Lean, by contrast, uses a much more expressive type system, powerful enough to encode and verify mathematical statements with complete rigor at compile time. This is what makes Lean particularly suitable as a **theorem prover** for formalizing mathematics.

Lean takes the idea of program derivation to its logical extreme: its type system is powerful enough that constructing programs from proofs of their specifications becomes the primary mode of interaction.

Lean's type system is based on **dependent type theory**, specifically the **Calculus of Inductive Constructions** (CIC) with various extensions [Car19]. It is important to note that **type theory** is not a single, unified theory, but rather a family of related theories with various extensions, ongoing developments, and rich historical ramifications.

Type theory emerged as an alternative foundation for mathematics, addressing paradoxes that arose in naive set theory. Consider Russell's famous paradox: let $S = \{x \mid x \notin x\}$ be the set of all sets that do not contain themselves. This construction is paradoxical, leading to the contradiction $S \in S \iff S \notin S$. Type theory resolves such issues by organizing objects into a stratified hierarchy of **types** (called **universes**): terms belong to types, types themselves belong to higher-level types, and so on. This stratification prevents self-referential constructions like

Russell's set, because a type at one level cannot refer to itself as an element at the same level.

Dependent type theory, the framework underlying Lean, extends basic type systems by allowing types to depend on values. For instance, one can define the type of vectors of length n, where n is a value (a natural number). This capability makes dependent type theory particularly expressive for formalizing mathematics.

Various proof assistants have been developed based on different variants of type theory, including Agda, Coq, Idris, and Lean. Each system makes different design choices regarding which rules and features to include. Lean's type system is based on the **Calculus of Inductive Constructions** (CIC), a variant of dependent type theory also used (with some differences) by the Coq proof assistant. CIC extends the simpler Calculus of Constructions with **inductive types**, which allow the definition of datatypes such as natural numbers, lists, and trees by specifying their constructors. A key type-theoretic distinction between Lean and Coq is that Lean additionally supports **quotient types** as a built-in primitive, which facilitates working with equivalence relations (see Section 4.1). For a detailed formal account of Lean's type theory, including its treatment of universes, definitional equality, and the kernel's trusted code base, we refer the reader to [Car19].

A fundamental design feature of Lean is its type structure, where **Prop** (the type of propositions) plays a special role and exhibits **proof irrelevance**. Proof irrelevance means that all proofs of a given proposition are treated as definitionally equivalent in type theory. What matters is simply whether a proposition is provable, not which specific proof is provided. The idea of distinguishing propositions from data types has a long history, going back to de Bruijn's AUTOMATH system (1967); see [Tho99], Section 9.1.4.

For our purposes, we do not need to delve deeply into the theoretical foundations; instead, we will introduce the relevant concepts as needed while working with Lean.

1.1 First Steps in Lean

In the language of type theory, and by extension in Lean, we write `x : X` to mean that x is a **term** of type X. For example, `2 : ℕ` annotates 2 as a natural number, or more precisely, as a term of the natural number type. Lean has internally defined types such as `Nat` or ℕ (you can type `\Nat` to get the Unicode symbol). The command `#check` allows us to inspect the type of any expression, term, or variable.

Example 1.1.1

```
#check 2 -- 2 : Nat
#check 2 + 2 -- 2 + 2 : Nat
```

[Try this example in Lean Web Editor] By following the link, you can try out the code in your browser. Lean provides a dedicated ***infoview*** *panel on the right side. Position your cursor after* `#check 2`*, and the infoview will display the output* `2 : Nat`*. This dynamic interaction, where the infoview responds to your cursor position, is what makes Lean an* ***interactive theorem prover****. As you move through your code, the infoview continuously updates, showing computations, type information, and proof states at each location.*

At first glance, one might be tempted to view the colon notation as analogous to the membership symbol $\in$ from set theory, treating types as if they were sets. While this intuition can be helpful initially, type theory offers a fundamentally richer perspective. The crucial insight is the **Curry–Howard correspondence**, also known as the **propositions-as-types** principle (we develop this correspondence in detail in Section 2.3). This correspondence establishes a deep connection between mathematical proofs and programs: **propositions correspond to types**, and **proofs correspond to terms** inhabiting those types. Under this interpretation, a term $x : X$ can also be understood as a **computational object**: x is a program or data structure of type X.

Lean is a concrete realization of the propositions-as-types principle: proving a theorem within the language amounts to constructing a term of the appropriate type. When we write `theorem_name : Proposition`, we are declaring that `theorem_name` is a proof (term) of `Proposition` (type). For example, consider proving that $2 + 2 = 4$.

Example 1.1.2

```
theorem two_plus_two_eq_four : 2 + 2 = 4 := rfl
```

Lean's syntax is designed to resemble the language of mathematics. Here, we use the `theorem` *keyword to encapsulate our proof, giving it the name* `two_plus_two_eq_four`*. This allows us to reference and reuse this result later in our code. After the colon,* `:`*, we state the proposition:* `2 + 2 = 4`*. The* `:=` *operator expects the proof term that establishes the theorem's validity. The proof itself consists of a single term:* `rfl` *(short for* ***reflexivity****). This is a proof term that works by* ***definitional equality****: Lean's kernel automatically reduces both sides of the equation to their normal form and verifies they are identical. Since* $2 + 2$ *computes to* 4*, the proof succeeds immediately.*

We can now use this theorem in subsequent proofs. For instance:

```
example : 1 + 1 + 1 + 1 = 4 := two_plus_two_eq_four
```

This works because Lean's kernel reduces both `1 + 1 + 1 + 1` *and* `4` *to the same normal form, so* `two_plus_two_eq_four` *(which is definitionally a proof that* `4 = 4`*) is accepted. Of course,* `rfl` *would work equally well here.*

The keyword `example` *is used to define anonymous expressions for demonstration purposes. Before diving into the discussion, here is another keyword,* `def`*, used to introduce definitions and functions:*

```
def addOne (n : Nat) : Nat := n + 1
```

This definition expects a natural number as its parameter, written `(n : Nat)`*, and returns a natural number.*

In the next chapter, we examine how the Curry–Howard correspondence is realized concretely in Lean, showing how each logical connective corresponds to a type-theoretic construction.

Chapter 2

Logic and Propositions as Types

In this chapter, we develop the logical and type-theoretic foundations that underpin Lean. We begin with propositional logic and its formalization via natural deduction (Section 2.1). We then show how types are constructed using analogous inference rules (Section 2.2). In Section 2.3, we make the connection explicit via the Curry–Howard correspondence. Section 2.4 extends the picture to predicate logic and dependent types, and Section 2.5 discusses the distinction between constructive and classical reasoning.

2.1 Propositional Logic and Natural Deduction

Logic is the study of reasoning, branching into various systems. We refer to **classical logic** as the logical system that underpins much of traditional mathematics. We first introduce **propositional logic**, the simplest form of classical logic. Later, in Section 2.4, we extend this to **predicate (or first-order) logic**, which adds **predicates** and **quantifiers**.

In propositional logic, a **proposition** is a statement that is either true or false, and a **proof** is a logical argument that establishes the truth of a proposition. Propositions can be combined with logical **connectives** such as "and" ($\wedge$), "or" ($\vee$), "not" ($\neg$), "false" ($\bot$), "true" ($\top$), "implies" ($\Rightarrow$), and "if and only if" ($\Leftrightarrow$). These connectives allow the creation of compound propositions. Here is how some connectives are represented in Lean:

Example 2.1.1 (Logical connectives in Lean)

```
#check @And  -- And : Prop → Prop → Prop
#check @Or   -- Or  : Prop → Prop → Prop
```

`Prop` is the type of propositions, introduced in Chapter 1.

Logic is often formalized through a framework known as the **natural deduction**

system, developed by Gentzen in the 1930s ([Wad15]). This approach specifies rules for deriving **conclusions** from **premises**, called **inference rules**.

Example 2.1.2 (Inference rule (schematic)) *Here is a schematic inference rule ([NPS90], page 35):*

$$\frac{P_1 \qquad P_2 \qquad \cdots \qquad P_n}{C}$$

The propositions $P_1, P_2, \ldots, P_n$ above the line are the premises, and the conclusion C appears below the line.

The inference rules we need are **introduction rules**, which specify how to form compound propositions from simpler ones, and **elimination rules**, which allow us to extract information from compound propositions. We now present the rules for the main connectives, following [Tho99], Section 1.1.

Conjunction ($\wedge$)

Introduction Rule. If we have proofs of both A and B, we can form a proof of $A \wedge B$:

$$\frac{A \qquad B}{A \wedge B}\ \wedge\text{-Intro}$$

Elimination Rules. From a proof of $A \wedge B$, we can extract a proof of either conjunct:

$$\frac{A \wedge B}{A}\ \wedge\text{-Elim}_1 \qquad\qquad \frac{A \wedge B}{B}\ \wedge\text{-Elim}_2$$

Disjunction ($\vee$)

Introduction Rules. A proof of either A or B suffices to form a proof of $A \vee B$:

$$\frac{A}{A \vee B}\ \vee\text{-Intro}_1 \qquad\qquad \frac{B}{A \vee B}\ \vee\text{-Intro}_2$$

Elimination Rule (proof by cases). If C follows from A and also from B, then C follows from $A \vee B$:

$$\frac{A \vee B \qquad [A] \vdash C \qquad [B] \vdash C}{C}\ \vee\text{-Elim}$$

Implication ($\rightarrow$)

Introduction Rule. To prove $A \rightarrow B$, we assume A and derive B:

$$\frac{[A] \vdash B}{A \rightarrow B}\ \rightarrow\text{-Intro}$$

Elimination Rule (modus ponens). Given a proof of $A \to B$ and a proof of A, we obtain a proof of B:

$$\frac{A \to B \qquad A}{B} \to\text{-Elim}$$

Notation 2.1.3 *We use $A \vdash B$ (called the **turnstile**) to denote a derivation of B from A; it can be read as "A entails B." The square brackets around a premise, as in $[A]$, indicate that the assumption A is **discharged** (i.e., released) at the conclusion of the rule. To illustrate: the $\to$-Intro rule says that to prove $A \to B$, we temporarily assume A, derive B under this assumption, and then* discharge *the assumption—meaning that the conclusion $A \to B$ no longer depends on A being true. The assumption was only needed during the derivation and is consumed by the rule.*

Having introduced the rules of propositional logic, we now turn to the type-theoretic side, where analogous rules govern the construction of types.

2.2 Primitive Types

Type theory employs a similar system of inference rules, but expressed in terms of **type judgments**. A type judgment has the form $\Gamma \vdash t : T$, meaning: under **context** Γ (a list of typed variables), the term t has type T. Introduction and elimination rules are then inference rules that derive new type judgments from existing ones, allowing us to construct new compound types from existing ones.

Example 2.2.1 (Type judgment rule (schematic)) *A schematic type judgment rule has the following form:*

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B}{\Gamma \vdash c : C}$$

Reading: if, under context Γ, we have a term a of type A and a term b of type B, then we can construct a term c of type C.

Technically, there are two additional kinds of rules that we will not treat in detail here: **formation rules**, which declare that a type is well-defined, and **computation rules**, which specify how terms are evaluated.

Moreover, there is a specific form of judgment for equality: $\Gamma \vdash A \equiv B$ type means "types A and B are **judgmentally (or definitionally) equal** in context Γ." Similarly for terms: $\Gamma \vdash t_1 \equiv t_2 : A$ means "terms t_1 and t_2 are judgmentally equal of type A in context Γ."

Brief explanation of equality in type theory. It is important to distinguish two notions of equality in type theory.

Definitional (or judgmental) equality is a relation that Lean's kernel can verify automatically, without any additional proof steps. Informally, two terms are definitionally equal when the kernel can reduce them to the same normal form by applying computation rules and unfolding definitions. A **normal form** is the most reduced state of a term, obtained by systematically applying reduction rules until no further reductions are possible. For example, $2 + 2$ and 4 are definitionally equal because both reduce to the same normal form.

In Lean, the operator `:=` is a **syntactic assignment**: it creates a new definition by binding a name to a term. For instance, writing `def x : Nat := 2 + 2` introduces a new name `x` and makes `x` definitionally equal to `2 + 2` (and hence to `4`). Note that `:=` is not a predicate that can be evaluated as true or false; it is part of the syntax for creating definitions.

In contrast, **propositional equality** is a statement *within* the logic (expressed as a type, e.g., `a = b`) that may require an explicit proof. Because it is a logical proposition rather than a computation, propositional equality is not directly decidable by the type checker and must be established by the user.

Let us now construct new types from given types A and B.

Product Type

The type $A \times B$ denotes the type of pairs (a, b) where $a : A$ and $b : B$, called the **product type**.

Introduction Rule (pairing).

$$\frac{a : A \qquad b : B}{(a, b) : A \times B}$$

In Lean:

```
Prod.mk a b : Prod A B    -- or A × B
(a, b) : A × B
⟨a, b⟩ : A × B
```

Elimination Rules (projections).

$$\frac{p : A \times B}{\mathsf{fst}(p) : A} \qquad \frac{p : A \times B}{\mathsf{snd}(p) : B}$$

In Lean:

```
p.1 : A        -- or Prod.fst p
p.2 : B        -- or Prod.snd p
```

Sum Type

The **sum type** $A + B$ (also called a coproduct or disjoint union) consists of values that are either of type A (tagged with inl) or of type B (tagged with inr).

Introduction Rules (injections).

$$\frac{a : A}{\mathsf{inl}(a) : A + B} \qquad \frac{b : B}{\mathsf{inr}(b) : A + B}$$

In Lean:

```
Sum.inl a : Sum A B    -- or A ⊕ B
Sum.inr b : Sum A B
```

Elimination Rule (case analysis).

$$\frac{p : A + B \qquad f : A \to C \qquad g : B \to C}{\mathsf{cases}(p, f, g) : C}$$

In Lean:

```
example (p : Sum A B) (f : A → C) (g : B → C) : C := by
  cases p with
  | inl x => exact f x
  | inr y => exact g y
```

Function Types

The type $A \to B$ represents functions from A to B.

Introduction Rule (lambda abstraction).

$$\frac{x : A \vdash \Phi : B}{\lambda x.\, \Phi : A \to B}$$

Here $\lambda x.\, \Phi$ is a function that maps any element $x : A$ to the element $\Phi : B$. In Lean, lambda abstraction is written using `fun` or λ:

```
def identityFun (A : Type) : A → A := fun x => x
```

Elimination Rule (function application).

$$\frac{f : A \to B \qquad a : A}{f(a) : B}$$

In Lean, function application is written by juxtaposition:

```
example (f : A → B) (a : A) : B := f a
```

Functions are a primitive concept in type theory. We can **apply** a function $f : A \to B$ to an element $a : A$ to obtain an element of B, denoted $f(a)$. In type theory, it is common to omit the parentheses and write the application simply as $f\ a$.

Having introduced the basic type constructions, we now make explicit the correspondence between these types and the logical connectives from Section 2.1.

2.3 The Curry–Howard Correspondence

The reader will have noticed a strong similarity between the inference rules for logical connectives (Section 2.1) and the rules for constructing types (Section 2.2). For instance, function types mirror implications, product types mirror conjunctions, and sum types mirror disjunctions. This is not a coincidence: it is an instance of the **Curry–Howard correspondence**, a deep connection between logic and computation discovered independently by Haskell Curry (1930s–50s) and William Howard (1969). It establishes a systematic dictionary between proofs and programs ([Tho99]):

Logic		**Type Theory**
Implication $P \Rightarrow Q$	$\longleftrightarrow$	Function type $P \to Q$
Conjunction $P \wedge Q$	$\longleftrightarrow$	Product type $P \times Q$
Disjunction $P \vee Q$	$\longleftrightarrow$	Sum type $P + Q$
Proposition	$\longleftrightarrow$	Type
Proof	$\longleftrightarrow$	Term

A proof of an **implication** $P \Rightarrow Q$ is a function that transforms any proof of P into a proof of Q. A proof of a **conjunction** $P \wedge Q$ consists of a pair containing proofs of both conjuncts. A proof of a **disjunction** $P \vee Q$ is either a proof of P or a proof of Q.

Lean realizes this correspondence concretely. For instance, $A \wedge B$ can be represented as `And A B` or `A ∧ B`. Its introduction rule is `And.intro _ _` or simply `⟨_, _⟩` (underscores are placeholders for the two proofs). The compound proposition $A \wedge B$ can then be decomposed using the elimination rules `And.left` and `And.right`.

Example 2.3.1 *Let us look at a simple Lean example:*

```
example {a b : Prop} (ha : a) (hb : b) : (a ∧ b) := And.intro ha hb
```

Using curly brackets `{ }`, we let Lean infer that `a` and `b` are propositions (`Prop`). The example states that given a proof of `a` (`ha`) and a proof of `b` (`hb`), we can form a proof of `a ∧ b`. The type of `And.intro` is:

```
And.intro : a → b → a ∧ b
```

It says: given a proof of a *and a proof of* b*, it returns a proof of* $a \wedge b$*. Here is an equivalent way to write the same statement using anonymous constructor notation:*

```
example {a b : Prop} (ha : a) (hb : b) : a ∧ b := ⟨ha, hb⟩
```

For a more concrete example, let us see how building a proof tree in natural deduction corresponds to constructing a term in Lean. To prove a compound statement, one typically follows a **top-down** approach, decomposing the goal step by step.

Example 2.3.2 (Associativity of Conjunction) *We prove that* $(A \wedge B) \wedge C$ *implies* $A \wedge (B \wedge C)$*. First, from the assumption* $(A \wedge B) \wedge C$*, we can derive* A*:*

$$\dfrac{\dfrac{(A \wedge B) \wedge C}{A \wedge B}\,\wedge E_1}{A}\,\wedge E_1$$

Second, we can derive $B \wedge C$*:*

$$\dfrac{\dfrac{\dfrac{(A \wedge B) \wedge C}{A \wedge B}\,\wedge E_1}{B}\,\wedge E_2 \qquad \dfrac{(A \wedge B) \wedge C}{C}\,\wedge E_2}{B \wedge C}\,\wedge I$$

Finally, combining these derivations we obtain $A \wedge (B \wedge C)$*:*

$$\dfrac{(A \wedge B) \wedge C \vdash A \qquad (A \wedge B) \wedge C \vdash B \wedge C}{A \wedge (B \wedge C)}\,\wedge I$$

Example 2.3.3 (Lean Implementation) *Let us now implement the same proof in Lean.*

```
theorem and_associative {a b c : Prop} : (a ∧ b) ∧ c → a ∧ (b ∧ c) :=
  fun h : (a ∧ b) ∧ c ↦
  -- First, from the assumption (a ∧ b) ∧ c, we can derive a:
  have hab : a ∧ b := h.left
  have ha : a := hab.left
  -- Second, extract b and c:
  have hc : c := h.right
  have hb : b := hab.right
  -- Finally, combining these derivations we obtain a ∧ (b ∧ c):
  show a ∧ (b ∧ c) from ⟨ha, ⟨hb, hc⟩⟩
```

We introduce the `theorem` *with the name* `and_associative`*. The type signature* `(a ∧ b) ∧ c → a ∧ (b ∧ c)` *represents our logical implication. We construct the proof of the implication as a function (following the Curry–Howard correspondence) using the* `fun` *keyword. The* `have` *keyword introduces local lemmas within our proof, allowing*

us to break down complex reasoning into manageable intermediate steps (mirroring the natural deduction proof tree from before).

Just before the `show` *keyword, the infoview displays the following context and goal:*

```
a b c : Prop
h : (a ∧ b) ∧ c
hab : a ∧ b
ha : a
hc : c
hb : b
⊢ a ∧ b ∧ c
```

Resembling type judgments, the goal appears after the turnstile (⊢), and the hypotheses above it form the current context.

Finally, `show a ∧ (b ∧ c) from ⟨ha, ⟨hb, hc⟩⟩` *asserts that we are constructing a proof of* `a ∧ (b ∧ c)` *using the term* `⟨ha, ⟨hb, hc⟩⟩`*. The* `show` *keyword improves readability and checks that the stated type matches the current goal. Here, the goal displayed as* `⊢ a ∧ b ∧ c` *and the statement* `a ∧ (b ∧ c)` *are* syntactically identical*, because ∧ is right-associative: no reduction rules are needed. If we had used* `show` *with a type that was only* propositionally *equal to the goal (but not definitionally equal), Lean would reject it.*

We have now seen the Curry–Howard correspondence for propositional connectives. To capture more complex mathematical ideas, we need to extend the picture to predicates and quantifiers.

2.4 Predicate Logic and Dependent Types

To capture more complex mathematical ideas, we extend our system from propositional logic to **predicate logic**. A **predicate** is a proposition that depends on a variable. In propositional logic we represent a proposition simply by P. In predicate logic, a predicate is written as $P(a)$, where a is a variable. Notice that a predicate is simply a function from elements to propositions.

This extension allows us to introduce **quantifiers**: ∀ ("for all") and ∃ ("there exists"). These quantifiers express that a given property holds either for every element or for at least one element, respectively. In Lean, if α is any type, we can represent a predicate `P` on α as an object of type $\alpha \to$ `Prop`. Thus, given `x : α`, the expression `P x : Prop` represents the proposition that P holds for x.

We can give an informal reading of the quantifiers as generalized logical operations:

$$\forall x.\, P(x) \;\sim\; P(a) \wedge P(b) \wedge P(c) \wedge \ldots$$
$$\exists x.\, P(x) \;\sim\; P(a) \vee P(b) \vee P(c) \vee \ldots$$

The dot symbol following the quantifier, as in $\forall x.$, binds every occurrence of the variable x in the expression $P(x)$. The expression $\forall x.\, P(x)$ can be understood as a generalized conjunction: P holds for all possible values of x. Similarly, $\exists x.\, P(x)$ is a generalized disjunction: P holds for at least one value of x.

Under the Curry–Howard correspondence, universal quantification corresponds to **dependent function types** (also called Pi types, written Π), while existential quantification corresponds to **dependent pair types** (also called Sigma types, written Σ). These are constructs from dependent type theory, in which types may depend on values.

In Lean, $\forall$ is the built-in dependent function type (Pi type), while $\exists$ is defined as an inductive type. We now present the Lean syntax for working with quantifiers directly.

Example 2.4.1 (Quantifiers in Lean) *Lean expresses quantifiers as follows:*

```
variable (X : Type) (P : X → Prop)
#check (∀ (x : X), P x)  -- Prop (corresponds to Pi type Π)
#check (∃ (x : X), P x)  -- Prop (corresponds to Sigma type Σ)
```

Example 2.4.2 (Universal introduction in Lean) *The **universal introduction rule** allows us to prove $\forall x,\, P(x)$ by proving $P(x)$ for an **arbitrary** x. In Lean, this corresponds to constructing a dependent function:*

```
example : ∀ n : Nat, n ≥ 0 :=
  fun n => Nat.zero_le n
```

Here `Nat.zero_le` is a built-in lemma proving that $0 \leq n$ for every natural number n.

Example 2.4.3 (Universal elimination in Lean) *The **universal elimination rule** allows us to instantiate a universally quantified statement with a specific value. In Lean, this is simply function application:*

```
example (h : ∀ n : Nat, n ≥ 0) : 5 ≥ 0 :=
  h 5
```

Example 2.4.4 (Existential introduction in Lean) *To prove an existential statement, we provide a* ***witness*** *together with a proof that the witness satisfies the predicate:*

```
example (x : Nat) (h : x > 0) : ∃ y, y < x :=
  ⟨0, h⟩
```

The term `⟨0, h⟩` *is a dependent pair holding the witness* (0) *and a proof that* $0 < x$.

Example 2.4.5 (Existential elimination in Lean) *The* ***existential elimination rule*** *allows us to use an existential hypothesis* $\exists x,\ P(x)$ *by extracting a witness and the associated proof. In Lean, this can be done in term mode using pattern matching via* `match`*:*

```
example (h : ∃ n : Nat, n + n = 10) : ∃ m : Nat, 10 = m + m :=
  match h with
  | ⟨n, hn⟩ => ⟨n, hn.symm⟩
```

The `match` *expression destructs the existential hypothesis* `h` *into a witness* `n : Nat` *and a proof* `hn : n + n = 10`*. We then construct the new existential proof with the same witness* `n` *and the reversed equality* `hn.symm : 10 = n + n`*. This mirrors the logical pattern: given* $\exists n,\ n + n = 10$*, we produce* $\exists m,\ 10 = m + m$ *using the same witness.*

We have now extended the Curry–Howard correspondence from propositional connectives to quantifiers. In the next section, we discuss an important foundational distinction: the difference between constructive and classical reasoning, and how Lean navigates between the two.

2.5 Constructive and Classical Reasoning

A common pairing in the foundations of mathematics is: *set theory with classical logic* on one side, and *type theory with constructive logic* on the other. While this pairing is the most common in practice, it is important to note that neither association is inherent. Intuitionistic set theories (such as IZF) exist, and on the other hand, Russell and Whitehead's type theory in *Principia Mathematica* (historically the first type theory) is entirely classical. Lean's type theory is constructive by default, but classical axioms can be invoked when needed.

A key point of departure between classical and constructive logic concerns the **law of excluded middle** (LEM): the principle that for every proposition P, either P or $\neg P$ holds. In classical logic, LEM is taken as an axiom, so every proposition is either true or false regardless of whether we can prove which. In constructive logic, asserting $P \vee \neg P$ requires actually producing either a proof of P or a proof of $\neg P$.

For an open conjecture such as the Riemann Hypothesis, where neither a proof nor a disproof is currently known, the constructive mathematician cannot assert this disjunction.

Constructive logic emphasizes that a statement is considered true only if we can explicitly provide a **witness** or **construction** for it. This is what makes constructive mathematics inherently **computable**: a proof of $\exists x,\ P(x)$ must include a concrete x together with a proof that $P(x)$ holds.

Proof by contradiction. It is important to distinguish two forms of reasoning that are often both called "proof by contradiction":

1. **Proof of negation** (constructively valid): to prove $\neg P$, assume P and derive a contradiction ($\bot$). This is simply the definition of $\neg P$ as $P \to \bot$ and is fully accepted in constructive logic.

2. **Classical contradiction**: to prove P, assume $\neg P$ and derive a contradiction, then conclude P. This requires LEM and is *not* constructively valid.

Example 2.5.1 (Constructive existence proof) *We give a constructive proof in Lean that there exist natural numbers a and b such that $a + b = 7$:*

```
example : ∃ a b : Nat, a + b = 7 := ⟨3, 4, rfl⟩
```

To prove the existential statement, we provide witnesses ($a = 3$ and $b = 4$) and a proof that the predicate holds ($3 + 4 = 7$, verified by `rfl`).

Ex falso quodlibet. The principle **ex falso quodlibet** (from a contradiction, anything follows) is *fully valid in constructive logic*. It corresponds to the elimination rule for the empty type $\bot$: if we have a proof of $\bot$ (falsehood), we can derive any proposition. In Lean, the `exfalso` tactic applies this principle:

Example 2.5.2 (Reasoning from false) *Here is an example of deriving any proposition from a contradiction:*

```
example (p : Prop) (h : False) : p := by
  exfalso
  exact h
```

The `exfalso` tactic transforms the goal into $\vdash \mathsf{False}$, meaning we must now produce a proof of falsehood. Since we already have $h : \mathsf{False}$ as a hypothesis, we conclude with `exact h`.

Classical reasoning in Lean. When genuinely classical reasoning is required, Lean provides the tactic `by_contra`, which invokes the law of excluded middle (available in Lean as `Classical.em`). This tactic allows us to assume $\neg P$ and derive a contradiction in order to conclude P:

Example 2.5.3 (Classical proof by contradiction)

```
example (p : Prop) (h : ¬¬p) : p := by
by_contra hnp
exact h hnp
```

The tactic `by_contra hnp` *introduces the hypothesis* `hnp : ¬p` *and changes the goal to* $\vdash$ False. *We then apply the double negation hypothesis* `h` *to* `hnp` *to obtain the required contradiction. This proof is not constructively valid since it relies on the law of excluded middle.*

In this chapter, we have seen how logical connectives and quantifiers correspond to type-theoretic constructions via the Curry-Howard correspondence, and how Lean blends constructive and classical reasoning. In the next chapter, we turn to how mathematical objects—from number systems to algebraic structures—are modeled in Lean and Mathlib.

Chapter 3

Structures, Type Classes, and Coercions

In this chapter, we explore how mathematical objects and their properties are represented in Lean and Mathlib. We begin with polymorphism and implicit arguments, then examine how Lean models rational numbers using structures (Section 3.1) and coercions (Section 3.2). Along the way, we introduce type classes as a mechanism for overloading operations across different types.

It is interesting to note that a relation, in type theory, can be expressed as a function: `R : α → α → Prop`. Similarly, when defining a predicate (`P : α → Prop`) we must first declare `α : Type` to be some arbitrary type. This is what is called **polymorphism**, more specifically **parametric polymorphism**. A canonical example is the identity function, written as $\alpha \to \alpha$, where α is a type variable. It has the same type for both its domain and codomain, meaning it can be applied to booleans (returning a boolean), numbers (returning a number), functions (returning a function), and so on.

In the same spirit, we can define a transitivity property of a relation, for a given type α, as follows:

```
def Transitive (α : Type) (R : α → α → Prop) : Prop :=
  ∀ x y z, R x y → R y z → R x z
```

To use `Transitive`, we must provide both the type α and the relation itself. For example, here is a proof of transitivity for the less-than-or-equal relation on $\mathbb{N}$ (in Lean, `Nat` or $\mathbb{N}$):

```
theorem le_trans_nat : Transitive Nat (· ≤ · : Nat → Nat → Prop) :=
  fun x y z h1 h2 => Nat.le_trans h1 h2
```

Notice that we used a function to discharge the universal quantifiers required by transitivity. Looking at this code, we immediately notice that explicitly passing the type argument `Nat` is somewhat repetitive. Lean allows us to omit it by letting the

type inference mechanism fill it in automatically. This is achieved by using **implicit arguments** with curly brackets:

```
def Transitive {α : Type} (R : α → α → Prop) : Prop :=
  ∀ x y z, R x y → R y z → R x z
theorem le_trans_nat' : Transitive (· ≤ · : Nat → Nat → Prop) :=
  fun x y z h1 h2 => Nat.le_trans h1 h2
```

Let us now revisit the transitivity proof, but this time for the less-than-or-equal relation on the rational numbers (`Rat` or $\mathbb{Q}$):

```
import Mathlib

theorem rat_le_trans : Transitive (· ≤ · : Rat → Rat → Prop) :=
  fun _ _ _ h1 h2 => Rat.le_trans h1 h2
```

Here, `Rat.le_trans` is the transitivity lemma for $\leq$ on rational numbers, provided by Mathlib. We import Mathlib to access `Rat` and `le_trans`. Mathlib is the community-driven mathematical library for Lean, containing a large body of formalized mathematics under ongoing development. It is the de facto standard library for mathematical proofs in Lean ([Lea20]). We will explore it throughout this chapter. The underscores indicate unnamed variables that we do not reference later. If we had named them, say `x y z`, then `h1` would be a proof of `x ≤ y`, `h2` would be a proof of `y ≤ z`, and `Rat.le_trans h1 h2` produces a proof of `x ≤ z`. The `Transitive` definition is imported from Mathlib and similarly defined as before.

Example 3.0.1 *The code can be made more readable using* ***tactic mode****. In this mode, you use tactics (commands provided by Lean or defined by users) to carry out proof steps succinctly, avoid repetition, and automate common patterns. This often yields shorter, clearer proofs than writing the full term by hand.*

```
import Mathlib

theorem rat_le_trans : Transitive (· ≤ · : Rat → Rat → Prop) := by
  intro x y z hxy hyz
  exact Rat.le_trans hxy hyz
```

This proof performs the same steps, but is easier to read. Using `by` *we enter Lean's tactic mode. Move your cursor just after* `by`*. The goal is initially displayed as* `⊢ Transitive fun x1 x2 ↦ x1 ≤ x2`*. The tactic* `intro` *introduces variables and hypotheses into the context (deconstructing universal quantifiers and implications). Now position your cursor just before* `exact` *and observe the infoview again. The goal is now* `⊢ x ≤ z`*, with the context showing the variables and hypotheses introduced by the previous tactic. The* `exact` *tactic closes the goal by supplying a term that exactly*

matches it. You can hover over each tactic to see its definition and documentation.

Having seen how Lean uses polymorphism and implicit arguments, let us now look inside Mathlib to see how these lemmas are actually proved.

3.1 Exploring Mathlib (The Rat Structure)

In the previous examples we used predefined lemmas such as `Nat.le_trans` and `Rat.le_trans` to simplify the presentation. We can now dig into the implementation of these lemmas. Let us look at the source code of `Rat.le_trans`. The Mathlib 4 documentation website is at `https://leanprover-community.github.io/mathlib4_docs`, and the documentation for `Rat.le_trans` is at `https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/Order/Ring/Unbundled/Rat.html#Rat.le_trans`. Click the "source" link there to jump to the implementation in the Mathlib repository. In editors like VS Code, you can also jump directly to the definition (Ctrl+click; Cmd+click on macOS). Another way to inspect source code is by using `#print Rat.le_trans`.

```
variable (a b c : Rat)
lemma le_trans (hab : a ≤ b) (hbc : b ≤ c) : a ≤ c := by
  rw [Rat.le_iff_sub_nonneg] at hab hbc
  have := Rat.add_nonneg hab hbc
  simp_rw [sub_eq_add_neg, add_left_comm (b + -a) c (-b),
    add_comm (b + -a) (-b), add_left_comm (-b) b (-a),
    add_comm (-b) (-a), add_neg_cancel_comm_assoc,
    ← sub_eq_add_neg] at this
  rwa [Rat.le_iff_sub_nonneg]
```

The proof uses several tactics and lemmas from Mathlib. The `rw` (or `rewrite`) tactic is very common and mirrors the mathematical practice of rewriting using equalities. Given a proof `h : A = B` (or an iff `h : A ↔ B`), the tactic `rw [h]` replaces occurrences of `A` with `B` in the goal. Using `rw [...] at hyp` performs the rewrite in a specified hypothesis instead. In this case, we rewrite the hypotheses `hab` and `hbc` via the Mathlib lemma `Rat.le_iff_sub_nonneg`, which states that for any two rational numbers `x` and `y`, `x ≤ y` is equivalent to `0 ≤ y - x`. After rewriting, we have:

```
hab : 0 ≤ b - a
hbc : 0 ≤ c - b
```

The `have` tactic introduces an intermediate result. If you omit a name, Lean assigns it the default name `this`. From `hab` and `hbc` we derive that `b - a` and `c - b` are nonnegative, hence their sum is nonnegative:

```
this : 0 ≤ b - a + (c - b)
```

The most involved step uses `simp_rw` to simplify the expression via a sequence of existing Mathlib lemmas. The tactic `simp_rw` is a variant of `rw` that can rewrite inside binders (such as universal quantifiers $\forall x,\ P(x)$), whereas plain `rw` cannot. A related tactic is `simp`, which is more powerful: it automatically searches through a database of lemmas tagged as **simp lemmas** and applies them repeatedly to simplify the goal. After these simplifications we obtain:

```
this : 0 ≤ c - a
```

Clearly, the proof relies mostly on `Rat.add_nonneg`. Mathlib defines `Rat` as an instance of a linear ordered field, implemented via a normalized fraction representation: a pair of integers (numerator and denominator) with positive denominator and coprime components ([Lea25]). To achieve this, it uses a **structure**. In Lean, a structure is a dependent record type used to group together related fields as a single data type. Unlike ordinary records, the type of later fields may depend on the values of earlier ones. Defining a structure automatically introduces a constructor (usually `mk`) and projection functions that retrieve the values of its fields. Structures may also include proofs expressing properties that the fields must satisfy.

```
structure Rat where
  mk' ::
  num : Int
  den : Nat := 1
  den_nz : den ≠ 0 := by decide
  reduced : num.natAbs.Coprime den := by decide
```

To create a rational number in Mathlib, we use the `Rat.mk'` constructor, providing a numerator and denominator. The fields `den_nz` and `reduced` are proofs that the denominator is nonzero and that the numerator and denominator are coprime, respectively. These proofs are automatically generated by Lean's `decide` tactic, which can evaluate decidable propositions by computation. We will not delve into the type-theoretic formalization of decidability, as it lies somewhat outside the scope of standard mathematical practice. Roughly speaking, a proposition is decidable if Lean can run a computation that reduces the statement to `true` or `false`. We refer the interested reader to [NPS90, Section 21.4] for a deeper treatment.

Example 3.1.1 *Here is how we can define rational numbers in Lean:*

```
def half : Rat := Rat.mk' 1 2  -- or (1 / 2 : Rat)
def third : Rat := Rat.mk' 1 3 -- or (1 / 3 : Rat)
```

When working with rational numbers, or more generally with structures, we must provide the required proofs as arguments to the constructor (or Lean must be able to verify them automatically). For instance, `Rat.mk' 1 0` or `Rat.mk' 2 6`

would be rejected. In the case of rationals, Mathlib unfolds the definition through `Rat.numDenCasesOn`. This principle states that, to prove a property of an arbitrary rational number, it suffices to consider numbers of the form n/d with $d > 0$ and $\gcd(n, d) = 1$. This reduction allows Mathlib to transform proofs about $\mathbb{Q}$ into proofs about $\mathbb{Z}$ and $\mathbb{N}$, and then lift the result back to rationals.

Let us return to `Rat.add_nonneg`, which was the important lemma used in the proof of `Rat.le_trans`. We are going to construct our own simplified implementation of rational numbers, inspired by Mathlib's approach, and use it to prove the theorem. We follow the same convention used in Mathlib: project to natural numbers and integers first, then lift the result back to rationals.

Let us start by creating the structure:

```
import Mathlib

structure myPreRat where
  num : Int
  den : Nat
  den_pos : 0 < den
```

Notice the similarity with Mathlib's definition. We do not include the coprimality condition. The name `myPreRat` (for "pre-rational") will become clear in Section 4.1, where we construct proper rational numbers using quotient types.

Our goal is to prove `myPreRat.add_nonneg`. Before we can state it, we need to define the operations $\leq$, `+`, and `0` for `myPreRat`. Operations such as addition or comparison need to be defined for each type. This is achieved through **type classes**. Type classes provide a powerful and flexible way to specify properties and operations that can be shared across different types, a mechanism called **ad hoc polymorphism** ([WB89]). A standard example is overloaded multiplication: the same symbol `*` denotes multiplication of integers (e.g., `3 * 3`) and of floating-point numbers (e.g., `3.14 * 3.14`). Lean exposes type classes for common operations:

```
class Add (α : Type u) where
  add : α → α → α

class LE (α : Type u) where
  le : α → α → Prop
```

Type classes are, under the hood, structures whose fields describe operations. The important additional features of type classes are type inference and **instances**.

When we register a type as an instance of a type class, Lean will automatically apply the operations defined by the class. For example, we can register addition for naturals or integers:

```
instance : Add Nat where
  add := Nat.add

instance : Add Int where
  add := Int.add
```

We now define instances for `myPreRat`:

```
instance : LE myPreRat where
  le r₁ r₂ := r₁.num * ↑r₂.den ≤ r₂.num * ↑r₁.den

instance : Add myPreRat where
  add r₁ r₂ := {
    num := r₁.num * ↑r₂.den + r₂.num * ↑r₁.den,
    den := r₁.den * r₂.den,
    den_pos := Nat.mul_pos r₁.den_pos r₂.den_pos
  }

def zero : myPreRat := { num := 0, den := 1, den_pos := by decide }

instance : OfNat myPreRat 0 where
  ofNat := zero
```

Once these instances are defined, Lean automatically infers which operation to use when we write `a + b` or `a ≤ b` for values of type `myPreRat`. The ↑ symbol denotes a coercion from `Nat` to `Int` (explained in Section 3.2). With `OfNat` we tell Lean that, in a context expecting `myPreRat`, the literal `0` should be interpreted as our `zero` definition.

We can now state and prove our lemmas. We first need a helper lemma relating the ordering to the numerator:

```
namespace myPreRat

lemma nonneg_iff (r : myPreRat) : 0 ≤ r ↔ 0 ≤ r.num := by
  change 0 * r.den ≤ r.num * 1 ↔ _
  simp
```

This states that a pre-rational number is nonnegative if and only if its numerator is nonnegative. The `change` tactic unfolds our definition of ≤ for `myPreRat`: since `0` has `num = 0` and `den = 1`, the inequality `0 ≤ r` becomes `0 * r.den ≤ r.num * 1`. Then `simp` simplifies the arithmetic to obtain the equivalence with `0 ≤ r.num`.

We can now prove `add_nonneg`:

```
lemma add_nonneg (a b : myPreRat) : 0 ≤ a → 0 ≤ b → 0 ≤ a + b := by
  simp only [nonneg_iff]
  intro ha hb
  apply Int.add_nonneg
  · exact Int.mul_nonneg ha (Int.natCast_nonneg b.den)
  · exact Int.mul_nonneg hb (Int.natCast_nonneg a.den)

end myPreRat
```

The `namespace` keyword defines a self-contained module. Outside of its scope, one refers to `add_nonneg` as `myPreRat.add_nonneg`.

We first simplify using `nonneg_iff`, which transforms the goal to ⊢ `0 ≤ a.num → 0 ≤ b.num → 0 ≤ (a + b).num`. We then introduce the two hypotheses `ha : 0 ≤ a.num` and `hb : 0 ≤ b.num`. By our definition of addition, the numerator of `a + b` is `a.num * ↑b.den + b.num * ↑a.den`. Since the numerator is an integer, we can use lemmas from Mathlib. We `apply` the lemma `Int.add_nonneg`, which states that the sum of two nonnegative integers is nonnegative. The `apply` tactic works backwards: given a goal ⊢ `G` and a lemma `lemma : P → Q → G`, it replaces the goal with two new subgoals ⊢ `P` and ⊢ `Q`. We close the first subgoal with `Int.mul_nonneg ha (Int.natCast_nonneg b.den)`, where `ha` provides the nonnegativity of `a.num`, and `Int.natCast_nonneg b.den` provides the nonnegativity of the coercion of `b.den` from `Nat` to `Int`. The second subgoal follows symmetrically.

Here is the full proof: [link to Lean live]

In the proof above, we repeatedly used the ↑ symbol to convert between `Nat` and `Int`. Let us now explain this mechanism properly.

3.2 Coercions and Type Casting

When working with multiple numeric types such as ℕ, ℤ, and ℚ, we often need to translate values between them ([LM20]). Lean provides two related mechanisms:

- **Casting** refers to the explicit conversion of a value from one type to another, typically using functions like `Nat.cast` or `Int.cast`. These functions come with accompanying lemmas that preserve properties across type conversions, such as `Nat.cast_pos` (if $n > 0$ as a natural number, then its image is positive as an integer).

- **Coercion** is a mechanism by which Lean *automatically* inserts a cast when a type mismatch would otherwise occur. For example, if `x : ℕ` and `y : ℤ`, then in the expression `x + y`, Lean automatically coerces `x` to ℤ.

The notation ↑ makes a coercion explicit in the source code, serving as a visual marker that a type conversion is taking place. For instance, in our `LE` instance for `myPreRat`, the expression `↑r₂.den` coerces the denominator from `Nat` to `Int` so that it can be multiplied with the numerator. Expressions involving coercions can become cumbersome. To illustrate, consider `↑m + ↑n < (10 : ℤ)`, where `m n : ℕ` are cast to ℤ. The expected simplified form is `m + n < 10` as an inequality over ℕ, since `+`, `<`, and the numeral `10` are all polymorphic.

Lean provides the tactic family `norm_cast` to perform these simplifications automatically. The tactic `norm_cast` normalizes casts by pushing them outward and eliminating redundant coercions:

```
example (m n : ℕ) (h : ↑m + ↑n < (10 : ℤ)) : m + n < 10 := by
  exact_mod_cast h
```

Here `exact_mod_cast` is a variant of `exact` that automatically handles coercion simplification. A related tactic, `push_cast`, pushes casts *inward* toward the leaves of an expression, which is useful when one wants to reason within a single type:

```
example (m n : ℕ) : (↑(m + n) : ℤ) = ↑m + ↑n := by
  push_cast
```

Having established how Lean handles type conversions, we now have all the tools needed to address a fundamental problem with `myPreRat`: different representations of the same rational number are not identified as equal. In the next chapter, we resolve this using quotient types and discuss how Mathlib organizes its algebraic hierarchy using type classes.

Chapter 4

Quotients and Algebraic Hierarchy

In this chapter, we address a fundamental limitation of the `myPreRat` structure from the previous chapter. We resolve this using quotient types (Section 4.1), then step back to examine how Mathlib's algebraic hierarchy is built using the type class mechanism introduced earlier (Section 4.2).

4.1 Quotients

Let us look at what goes wrong with `myPreRat`:

```
example : myPreRat.mk 2 4 (by decide) ≠ myPreRat.mk 1 2 (by decide) := by
  simp
```

This example proves that `myPreRat.mk 2 4` and `myPreRat.mk 1 2` are not equal, even though mathematically $\frac{2}{4} = \frac{1}{2}$! The name `myPreRat` was already alluding to the need for further work.

In mathematics, we treat different representations of the same rational number as equivalent through an **equivalence relation** ([Alg19]). We consider fractions like $\frac{1}{2}$, $\frac{2}{4}$, and $\frac{3}{6}$ as belonging to the same equivalence class. More precisely, mathematics uses **quotients** to group elements of a set by an equivalence relation. For instance, the equivalence class $\left[\frac{1}{2}\right] = \left\{\ldots, \frac{-2}{-4}, \frac{1}{2}, \frac{2}{4}, \frac{3}{6}, \ldots\right\}$ represents all fractions equivalent to $\frac{1}{2}$. The set of rational numbers is the set of such equivalence classes.

Algebraically, each rational number can be represented as a pair of integers (a, b) where $b \neq 0$. This construction is formalized by an equivalence relation $\sim_{\mathbb{Q}}$:

$$\mathbb{Q} = \big(\mathbb{Z} \times (\mathbb{Z} \setminus \{0\})\big)/\sim_{\mathbb{Q}}$$

The equivalence relation is defined as:

$$(a, b) \sim_{\mathbb{Q}} (c, d) \iff ad = bc \quad \text{for all } (a, b), (c, d) \in \mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$$

The relation $\sim_{\mathbb{Q}}$ must satisfy reflexivity, symmetry, and transitivity. Moreover, each operation defined on the quotient set must be **well-defined**; that is, the result must not depend on the choice of representatives. If we naively define addition as

$$(a, b) + (c, d) = (a + c,\ b + d),$$

we encounter a problem. Consider:

$$(1, 2) + (1, 3) = (2, 5) \quad \text{and} \quad (1, 2) + (2, 6) = (3, 8).$$

But $(1, 3) \sim_{\mathbb{Q}} (2, 6)$ since $1 \cdot 6 = 2 \cdot 3$, yet $(2, 5) \not\sim_{\mathbb{Q}} (3, 8)$ since $2 \cdot 8 = 16 \neq 15 = 5 \cdot 3$. A well-defined definition for addition is instead:

$$(a, b) + (c, d) = (ad + bc,\ bd).$$

We will verify this in our formalization.

Similarly, in type theory and Lean we have **quotient types**, which allow us to construct new types by means of equivalence relations. We now define proper rational numbers using quotient types in Lean. As mentioned earlier, this differs from Mathlib's approach, which achieves the same goal by mechanically reducing each rational number to a canonical form using coprimality.

We start from the notion of equivalence relations. The structure `Equivalence` has fields `refl`, `symm`, and `trans`. You can inspect its definition using `#print Equivalence`. The `Setoid` type class encapsulates an equivalence relation on a given type, requiring that the relation is indeed an equivalence via its `iseqv` field.

We begin by defining the `Setoid` instance and the quotient type:

```
instance myRel : Setoid myPreRat where
  r p q := p.num * q.den = q.num * p.den
  iseqv := by
    constructor
    · intro p; rfl
    · rintro ⟨p, p', hp'⟩ ⟨q, q', hq'⟩
      simp [Eq.comm]
    · rintro ⟨p, p', hp'⟩ ⟨q, q', hq'⟩ ⟨r, r', hr'⟩ hpq hqr
      apply mul_left_cancel₀ (mod_cast hq'.ne' : q' ≠ (0 : ℤ))
      grind

abbrev myRat : Type := Quotient myRel
```

In the `iseqv` field, we prove reflexivity, symmetry, and transitivity of our relation. Reflexivity (`intro p; rfl`) is immediate. Symmetry uses `Eq.comm` to swap the sides of the equation. For transitivity, the key step involves canceling the common factor `q.den` using `mul_left_cancel₀` (left multiplication cancellation for integers), which

states that if $a \cdot b = a \cdot c$ and $a \neq 0$, then $b = c$. The expression `mod_cast hq'.ne' : q' ≠ (0 : ℤ)` takes `hq'` (which states `0 < q.den` as a natural number) and casts it to a proof that `q.den ≠ 0` as an integer. Finally, `abbrev` is a Lean keyword that creates a type abbreviation. Unlike `def`, which creates a new definition that must be unfolded explicitly, with `abbrev` Lean automatically treats `myRat` as `Quotient myRel` without requiring manual unfolding.

Before defining operations on `myRat`, we must prove that our operations are well-defined on the quotient, that is, that the result does not depend on the choice of representative. This is achieved using `Quotient.lift₂`: to define a function from a quotient type, we must provide an underlying function on the representatives and a proof that equivalent representatives yield the same result.

We first prove that the ordering respects the equivalence relation. The heart of the proof is the forward direction:

```
private theorem le_respects_equiv_forward
    (a₁ b₁ a₂ b₂ : myPreRat)
    (ha : a₁ ≈ a₂) (hb : b₁ ≈ b₂)
    (h : a₁ ≤ b₁) : a₂ ≤ b₂ := by
  have pos_prod : (0 : Int) < (a₁.den * b₁.den) :=
    myPreRat.den_prod_pos a₁ b₁
  have pos_prod2 : 0 < (a₂.den * b₂.den : Int) :=
    myPreRat.den_prod_pos a₂ b₂
  apply Int.le_of_mul_le_mul_right _ pos_prod
  calc (a₂.num * b₂.den) * (a₁.den * b₁.den)
      = a₂.num * a₁.den * b₂.den * b₁.den := by ring
    _ = a₁.num * a₂.den * b₂.den * b₁.den := by rw [← ha]
    _ = a₁.num * b₁.den * (a₂.den * b₂.den) := by ring
    _ ≤ b₁.num * a₁.den * (a₂.den * b₂.den) :=
        Int.mul_le_mul_of_nonneg_right h (Int.le_of_lt pos_prod2)
    _ = b₁.num * b₂.den * a₁.den * a₂.den := by ring
    _ = b₂.num * b₁.den * a₁.den * a₂.den := by rw [← hb]
    _ = (b₂.num * a₂.den) * (a₁.den * b₁.den) := by ring
```

Given $h : a_1 \leq b_1$ (meaning $a_1.\text{num} \cdot b_1.\text{den} \leq b_1.\text{num} \cdot a_1.\text{den}$) and equivalences $ha : a_1 \approx a_2$ and $hb : b_1 \approx b_2$, we need to prove $a_2 \leq b_2$. The strategy is to introduce a common positive factor and use the given information. We apply `Int.le_of_mul_le_mul_right`, which states that to prove $X \leq Y$, it suffices to prove $X \cdot Z \leq Y \cdot Z$ for positive Z, then cancel Z. We choose $Z = a_1.\text{den} \cdot b_1.\text{den}$ (shown positive by `pos_prod`). The `calc` block proceeds as follows: we rearrange the left side and substitute using ha, apply h (multiplying by the positive factor $a_2.\text{den} \cdot b_2.\text{den}$ to preserve the inequality), rearrange and substitute using hb, and finally rearrange to match the required form. After canceling the common factor, we obtain $a_2 \leq b_2$.

Throughout the proof, we use the `ring` tactic from Mathlib, which automates proofs of equalities in commutative rings (such as $\mathbb{Z}$).

The full well-definedness proof for ordering uses symmetry to avoid duplication:

```
theorem myRel_respects_le (a₁ b₁ a₂ b₂ : myPreRat) :
    a₁ ≈ a₂ → b₁ ≈ b₂ → (a₁ ≤ b₁) = (a₂ ≤ b₂) := by
  intro ha hb
  simp only [eq_iff_iff]
  constructor
  · exact le_respects_equiv_forward a₁ b₁ a₂ b₂ ha hb
  · exact fun h =>
      le_respects_equiv_forward a₂ b₂ a₁ b₁ ha.symm hb.symm h
```

We transform the equality of propositions into a biconditional using `eq_iff_iff`, then prove both directions with `constructor`. Since both goals are symmetric, we reuse `le_respects_equiv_forward`.

We also prove that addition is well-defined:

```
theorem myRel_respects_add (a₁ b₁ a₂ b₂ : myPreRat) :
    a₁ ≈ a₂ → b₁ ≈ b₂ → (a₁ + b₁) ≈ (a₂ + b₂) := by
  intro ha hb
  calc (a₁.num * b₁.den + b₁.num * a₁.den) * (a₂.den * b₂.den)
      = a₁.num * b₁.den * a₂.den * b₂.den
        + b₁.num * a₁.den * a₂.den * b₂.den := by ring
    _ = a₂.num * a₁.den * b₁.den * b₂.den
        + b₂.num * b₁.den * a₁.den * a₂.den := by rw [← ha, ← hb]; ring
    _ = (a₂.num * b₂.den + b₂.num * a₂.den) * (a₁.den * b₁.den)
        := by ring
```

We need to show that if $a_1 \approx a_2$ and $b_1 \approx b_2$, then $(a_1 + b_1) \approx (a_2 + b_2)$. By unfolding the addition definition and the relation, we must prove:

$$\begin{aligned}&(a_1.\text{num} \cdot b_1.\text{den} + b_1.\text{num} \cdot a_1.\text{den}) \cdot (a_2.\text{den} \cdot b_2.\text{den})\\ =\ &(a_2.\text{num} \cdot b_2.\text{den} + b_2.\text{num} \cdot a_2.\text{den}) \cdot (a_1.\text{den} \cdot b_1.\text{den})\end{aligned}$$

The `calc` proof proceeds in three steps. First, we distribute the product over the sum using `ring`. Next, we apply the equivalences ha and hb using `rw [← ha, ← hb]`, which substitutes $a_1.\text{num} \cdot a_2.\text{den}$ with $a_2.\text{num} \cdot a_1.\text{den}$ and similarly for b. Finally, we factor back into sum-times-product form using `ring`.

Now that we have established well-definedness, we can define operations on the quotient type `myRat`:

```
instance : LE myRat where
  le r₁ r₂ := Quotient.lift₂ (fun a b ↦ a ≤ b)
    myRel_respects_le r₁ r₂

instance : Add myRat where
  add r₁ r₂ := Quotient.lift₂ (fun a b ↦ ⟦a + b⟧)
    (fun a₁ b₁ a₂ b₂ ha hb ↦
      Quotient.sound (myRel_respects_add a₁ b₁ a₂ b₂ ha hb))
    r₁ r₂

instance : OfNat myRat 0 where
  ofNat := ⟦myPreRat.zero⟧

lemma add_nonneg (a b : myRat) :
    0 ≤ a → 0 ≤ b → 0 ≤ a + b := by
  induction a using Quotient.ind with | _ a =>
  induction b using Quotient.ind with | _ b =>
  intro ha hb
  exact myPreRat.add_nonneg a b ha hb
```

Notice how we reuse the previous `add_nonneg` lemma for the quotient type. The syntax `induction a using Quotient.ind with _ a =>` unwraps the quotient value `a : myRat` to its underlying representative `a : myPreRat`.

We have now constructed `myRat.add_nonneg`. However, our original discussion was about proving transitivity of the less-or-equal operator. In our earlier work with natural numbers, we used `Nat.le_trans`, a theorem specifically for natural numbers. However, the transitivity property holds not only for naturals but also for integers, reals, and any partially ordered set.

Rather than having each piece of downstream code call a different type-specific lemma (`Nat.le_trans`, `Rat.le_trans`, etc.), Mathlib provides a general lemma `le_trans` that works for any type endowed with a partial order. It is important to understand what this generality achieves: the general `le_trans` is not performing a clever argument—it is simply retrieving the transitivity proof that was supplied as part of the partial order structure. We still need to provide a new proof of transitivity whenever we create a new `PartialOrder` instance. The benefit is *downstream*: any code that uses transitivity can call the single lemma `le_trans` rather than needing to know which specific type it is working with. This provides a consistent interface for all ordered types.

Mathlib achieves this through type classes and a carefully constructed algebraic hierarchy. We already used type classes such as `Add` and `LE` to define operations on `myPreRat`. Knowing that rational numbers form a totally ordered set, we can enhance

`myRat` with the `LinearOrder` type class:

Remark 4.1.1 *The following code was developed and tested with Mathlib version 4.24.0. Later versions of Mathlib may introduce instances (such as `Quotient.instPreorder`) that conflict with the order structure defined here. This is a consequence of Mathlib's ongoing development and not a fundamental issue with the construction.*

```
noncomputable instance : LinearOrder myRat where
  le_refl p := by
    induction p using Quotient.ind with | _ a =>
    exact myPreRat.le_refl a

  le_trans p q r := by
    induction p using Quotient.ind with | _ a =>
    induction q using Quotient.ind with | _ b =>
    induction r using Quotient.ind with | _ c =>
    intro hab hbc
    exact myPreRat.le_trans a b c hab hbc

  le_antisymm p q := by
    induction p using Quotient.ind with | _ a =>
    induction q using Quotient.ind with | _ b =>
    intro hab hba
    exact Quotient.sound (myPreRat.le_antisymm a b hab hba)

  le_total p q := by
    induction p using Quotient.ind with | _ a =>
    induction q using Quotient.ind with | _ b =>
    cases Int.le_total (a.num * b.den) (b.num * a.den) with
    | inl h => exact Or.inl h
    | inr h => exact Or.inr h

  toDecidableLE := Classical.decRel _
end myRat
```

The structure follows the same pattern: use `Quotient.ind` to unwrap quotient values to their representatives, then apply the corresponding proof for `myPreRat`. The antisymmetry case uses `Quotient.sound`, which states that if two representatives are equivalent ($a \approx b$), then their equivalence classes are equal ($[\![a]\!] = [\![b]\!]$).

The underlying proofs for `myPreRat` are:

```
theorem le_refl (a : myPreRat) : a ≤ a := by
  exact Int.le_refl _

theorem le_trans (a b c : myPreRat) :
    a ≤ b → b ≤ c → a ≤ c := by
  intro hab hbc
  apply Int.le_of_mul_le_mul_right _ b.den_pos_int
  calc (a.num * c.den) * b.den
      = (a.num * b.den) * c.den := by ring
    _ ≤ (b.num * a.den) * c.den :=
        Int.mul_le_mul_of_nonneg_right hab
          (Int.le_of_lt c.den_pos_int)
    _ = (b.num * c.den) * a.den := by ring
    _ ≤ (c.num * b.den) * a.den :=
        Int.mul_le_mul_of_nonneg_right hbc
          (Int.le_of_lt a.den_pos_int)
    _ = (c.num * a.den) * b.den := by ring

theorem le_antisymm (a b : myPreRat) :
    a ≤ b → b ≤ a → a ≈ b := by
  intro hab hba
  exact Int.le_antisymm hab hba
```

Reflexivity and antisymmetry are straightforward applications of the corresponding integer properties. For transitivity, given $hab : a \leq b$ and $hbc : b \leq c$, we need to prove $a \leq c$. The strategy is to introduce a common positive factor b.den, prove the inequality with this factor, then cancel it using `Int.le_of_mul_le_mul_right`. The `calc` chain rearranges terms, applies hab and hbc (multiplying by positive factors to preserve the inequalities), and rearranges to match the required form.

The property `le_total` asserts that the order is total: for any a, b, either $a \leq b$ or $b \leq a$. By unwrapping the quotient, this reduces to an inequality of integers, and we apply `Int.le_total`.

The field `toDecidableLE := Classical.decRel _` requires special attention. In Mathlib, a `LinearOrder` is not merely a set of axioms; it also defines `min` and `max` functions. By default, `min a b` is defined as `if a ≤ b then a else b`. To evaluate this conditional, Lean requires the relation ≤ to be *decidable* (i.e., there must be an algorithm to determine whether it holds). While we could write an explicit comparison algorithm for our rationals, we opted for a concise approach using `Classical.decRel`, which invokes classical logic to treat every proposition as decidable. Because this relies on the axiom of choice rather than an executable algorithm, the resulting

functions cannot be computed by the kernel, necessitating the `noncomputable` marker.

Here is the full construction of `myRat`: [link to Lean live]

With the use of type classes such as `LinearOrder`, we enhanced `myRat` so that general theorems about linear orders apply to it automatically. In the next section, we examine this mechanism more carefully and discuss Mathlib's broader algebraic hierarchy.

4.2 Type Classes and Algebraic Hierarchy

Structures and type classes are heavily used in Mathlib for providing properties such as ordered rings, fields, metric spaces, and topological spaces, which we refer to as **algebraic structures**. An algebraic structure is defined by a carrier type and a set of operations and properties. We write $C\ \alpha$ where α is the **carrier type** (the underlying set of elements) and C is the **structure** (the operations and properties defined on that type). For example, in a group structure Group α, α is the carrier type (such as $\mathbb{Z}$ or $\mathbb{Q}$), and Group defines the multiplication, identity element, and inverse operation on α.

Note that this is a simplified picture: type classes can take multiple type arguments or additional non-type arguments. For instance, `Module` $\mathbb{Q}$ $\mathbb{R}$ takes two types ($\mathbb{Q}$ as the scalar ring and $\mathbb{R}$ as the module), and `OfNat` α `n` takes a type and a natural number.

Under the hood, a type class is a structure. The main difference between the two is that type classes use **instance resolution**: a mechanism that allows Lean to automatically infer properties about a type when the latter is declared as an instance of a type class. Using structures alone would require explicitly passing the structure as an argument to functions that need it, which is cumbersome.

Let us examine a simple example to see the main difference. Using a structure:

```
structure Semigroup' (α : Type*) where
  mul : α → α → α
  mul_assoc : ∀ a b c : α,
    mul (mul a b) c = mul a (mul b c)

def double {α : Type*} (s : Semigroup' α) (x : α) : α :=
  s.mul x x

def Semigroup'_Int : Semigroup' ℤ where
  mul := (· + ·)
  mul_assoc := Int.add_assoc

def Semigroup'_Rat : Semigroup' ℚ where
  mul := (· + ·)
  mul_assoc := Rat.add_assoc

#eval double Semigroup'_Int (-2)     -- -4
#eval double Semigroup'_Rat (1/2)    -- 1
```

The `double` function takes an explicit `Semigroup'` α argument to perform the doubling operation. When calling `double`, we must explicitly provide the semigroup instance, such as `Semigroup'_Int` or `Semigroup'_Rat`.

In contrast, using a type class:

```
class Semigroup'' (α : Type*) where
  mul : α → α → α
  mul_assoc : ∀ a b c : α,
    mul (mul a b) c = mul a (mul b c)

instance : Semigroup'' ℤ where
  mul := (· + ·)
  mul_assoc := Int.add_assoc

instance : Semigroup'' ℚ where
  mul := (· + ·)
  mul_assoc := Rat.add_assoc

def double' {α : Type*} [Semigroup'' α] (x : α) : α :=
  Semigroup''.mul x x

#eval double' (-2 : ℤ)     -- -4
#eval double' (1/2 : ℚ)    -- 1
```

Notice the square brackets `[Semigroup'' α]` in the definition of `double'`. This syntax indicates that `Semigroup'' α` is a type class constraint. When we call `double'`, we do not need to explicitly provide the semigroup instance. Instead, Lean automatically infers the appropriate instance based on the type of the argument. This is the power of instance resolution.

We chose the semigroup example because it is simple yet demonstrates how algebraic structures can be extended. A semigroup can be extended to a monoid or a group by adding more properties. Using structures and type classes, we can implement this inheritance pattern:

```
class Group'' (α : Type*) extends Semigroup'' α where
  one : α
  left_id : ∀ a : α, mul one a = a
  right_id : ∀ a : α, mul a one = a
  inv : α → α
  left_inv : ∀ a : α, mul (inv a) a = one
  right_inv : ∀ a : α, mul a (inv a) = one

instance : Group'' ℤ where
  -- omitted

instance : Group'' ℚ where
  -- omitted

theorem mul_cancel₀ {α : Type*} [Group'' α] (a b c : α)
    (h : Semigroup''.mul a b = Semigroup''.mul a c) :
    b = c := by sorry
```

Now we can state the cancellation law once and use it for any type that is an instance of `Group''`. We have already demonstrated this principle when discussing transitivity of the less-or-equal relation and when enhancing `myRat` with the `LinearOrder` structure.

Limitations of type classes. While type classes are powerful, they are not without drawbacks. Two important limitations are worth noting:

- **Performance.** Instance resolution can be slow, especially in large projects like Mathlib where the hierarchy of type classes is deep and highly interconnected. Lean must search through many candidate instances to find the right one, and this search can become expensive.

- **Conflicting instances.** If two different instances of the same type class are registered for the same type, the results can be unpredictable or lead

to subtle errors. For example, a type might carry two different orderings, but Lean's instance resolution will silently pick one, potentially leading to confusion. Mathlib takes great care to avoid such conflicts, but they can arise when combining independently developed libraries.

Mathlib's algebraic hierarchy. Mathlib builds an extensive algebraic hierarchy using type classes. This hierarchy is far from being a simple tree; there are different nuances and technical approaches that are beyond the scope of this thesis. For a detailed discussion on the design and implementation of Mathlib's algebraic hierarchy, we refer the reader to [Lea20].

In this chapter, we have seen how quotient types allow us to identify equivalent representations, turning `myPreRat` into the well-behaved type `myRat`. We then examined how Mathlib's algebraic hierarchy, built on type classes and instance resolution, provides a consistent and extensible framework for organizing mathematical structures. In the next chapter, we turn to how topology and analysis are formalized in Lean, building on the type class machinery introduced here.

Chapter 5

Topology in Lean

In this chapter, we examine how topology is formalized in Lean and Mathlib, building on the type class machinery introduced in the previous chapters. We begin by discussing how real numbers are represented in Mathlib and why many operations on them are noncomputable (Section 5.1). We then introduce Mathlib's `TopologicalSpace` type class and the definition of continuity in terms of open sets. In Section 5.2, we discuss filters (Mathlib's central tool for reasoning uniformly about limits, convergence, and continuity) and show how they connect global and local notions of continuity. Finally, in Section 5.3, we recall the definitions of connectedness and path-connectedness that will be needed in the next chapter.

5.1 Real Numbers and Topological Spaces

In the previous chapters, we built rational numbers from integers using quotient types. In Mathlib, real numbers are constructed by a similar process: they are defined as the **Cauchy completion** of $\mathbb{Q}$, that is, equivalence classes of Cauchy sequences of rational numbers [Lea25]. Unlike our construction of `myRat`, where classical axioms were used only for convenience (to avoid writing a decision procedure for inequality), the standard `Real` type relies on classical axioms more fundamentally. Many operations on real numbers, such as the sine function, the exponential, or division, are defined using the axiom of choice and do not correspond to executable algorithms. Consequently, they are marked as `noncomputable`:

```
noncomputable def realSin (x : ℝ) : ℝ := Real.sin x
```

This marker indicates that Lean's kernel cannot evaluate the function by computation, but we can still state and prove theorems about it.

Real numbers in Mathlib are registered as instances of numerous algebraic and topological type classes, including: linear ordered field, normed space, metric space, and topological space ([Lea20]). Since $\mathbb{R}$ is a metric space, notions like convergence

can be defined using the familiar ε-δ formulation:

```
def ConvergesTo (s : ℕ → ℝ) (a : ℝ) :=
  ∀ ε > 0, ∃ N, ∀ n ≥ N, |s n - a| < ε
```

where `s` is a sequence and `a` is the limit point. More generally, these concepts are formalized using topology.

5.1.1 The `TopologicalSpace` type class

In standard textbooks such as Munkres [Mun00], a topological space is defined as a set X equipped with a collection $\mathcal{T}$ of subsets of X, called **open sets**, satisfying the following axioms:

- The whole space X and the empty set $\varnothing$ are open.
- The intersection of any two open sets is open.
- The union of any collection of open sets is open.

This is reflected in Mathlib's `TopologicalSpace` type class:

```
class TopologicalSpace (α : Type*) where
  IsOpen : Set α → Prop
  isOpen_univ : IsOpen Set.univ
  isOpen_inter : ∀ (s t : Set α), IsOpen s → IsOpen t → IsOpen (s ∩ t)
  isOpen_sUnion : ∀ (S : Set (Set α)), (∀ t ∈ S, IsOpen t) → IsOpen
    (sUnion S)
```

Recall that Lean treats sets as predicates (`Set α := α → Prop`), where set membership is function application. Here, `IsOpen` is a predicate on sets indicating whether they are open. The three fields correspond to the standard topological axioms:

- The whole space is open (`isOpen_univ`).
- The intersection of two open sets is open (`isOpen_inter`).
- The union of any collection of open sets is open (`isOpen_sUnion`).

Note 5.1.1 *The empty set is open, as it is the union of the empty collection of open sets:* $\varnothing = \bigcup \varnothing$.

Using this type class, global continuity can be defined in terms of open sets, and indeed this is how it is defined in Mathlib:

```
structure Continuous (f : X → Y) : Prop where
  isOpen_preimage : ∀ s, IsOpen s → IsOpen (f ⁻¹' s)
```

A function $f : X \to Y$ between topological spaces is continuous if the preimage of every open set in Y is open in X. This is equivalent to the familiar ε-δ definition when X and Y are metric spaces.

However, when proving results about limits, convergence, and continuity, Mathlib predominantly uses an alternative framework based on **filters**. This approach may seem more abstract initially, but it provides a more general and powerful framework that works uniformly across all topological spaces and, by extension, metric spaces.

We now introduce filters and explain why Mathlib adopts them as the primary language for reasoning about convergence.

5.2 Filters

The concept of a limit is quite broad: there are many types of limits to consider. For instance, the limit of a sequence or a function at a point, limits at infinity (from above or below), one-sided limits (from the left or right), and so on. Defining each of these separately would require a huge amount of work in Mathlib, with significant duplication of theorems and proofs. Moreover, many fundamental theorems (like the characterization of continuity via limits) would need to be reproved for each type of limit. Fortunately, Bourbaki solved this problem by introducing the notion of **filters**, which unify all concepts of limits, convergence, neighborhoods, and terms like "eventually" or "frequently" into a single framework.

Intuitively, a filter represents a notion of "sufficiently large" subsets. More formally, a filter F on a type X is a collection of subsets of X satisfying three axioms:

```
structure Filter (α : Type*) where
  sets : Set (Set α)
  univ_sets : Set.univ ∈ sets
  sets_of_superset {x y : Set α} : x ∈ sets → x ⊆ y → y ∈ sets
  inter_sets {x y : Set α} : x ∈ sets → y ∈ sets → x ∩ y ∈ sets
```

The field `sets` suggests thinking of a filter as a collection of sets. The three axioms correspond to:

- $X \in F$ (the whole space is in the filter).
- If $U \in F$ and $U \subseteq V$, then $V \in F$ (supersets of "large" sets are "large").
- If $U, V \in F$, then $U \cap V \in F$ (finite intersections of "large" sets are "large").

Note that if a filter F contains the empty set, then it contains all subsets of X (since every set is a superset of $\varnothing$). Such a filter is called the **bottom filter** and is degenerate; non-trivial filters never contain the empty set.

5.2.1 Key filter concepts

Filters are quite categorical objects, and we will use them intuitively rather than developing the full theory. In particular, we will use the following concepts:

- **Neighborhood filter** $\mathcal{N}$ `x`: In a topological space, this filter contains all neighborhoods of the point x. A set is in $\mathcal{N}$ `x` if it contains an open set containing x. This captures the idea of "near x."
- **At-top filter** `atTop : Filter ℕ`: Contains sets that include all sufficiently large natural numbers. Formally, $U \in$ `atTop` if and only if there exists N such that $\{n \mid n \geq N\} \subseteq U$. This captures the idea of "$n \to \infty$."
- $\forall^f$ `x in f, p x` (`f.Eventually p`): "Eventually in filter f, property p holds." This means there exists some set $U \in f$ such that p holds for all $x \in U$.
- $\exists^f$ `x in f, p x` (`f.Frequently p`): "Frequently in filter f, property p holds." This means for every set $U \in f$, there exists some $x \in U$ where p holds. This captures the idea that p holds "infinitely often" or "arbitrarily close."
- `Tendsto f l₁ l₂`: "Function f tends from filter l_1 to filter l_2." This is used for convergence.

5.2.2 The `Tendsto` predicate

The predicate `Tendsto` is the key definition that makes filters useful for convergence. Its definition is remarkably concise:

```
def Tendsto (f : α → β) (l₁ : Filter α) (l₂ : Filter β) : Prop :=
  l₁.map f ≤ l₂
```

where `Filter.map f l₁` is the **pushforward** of the filter l_1 along f: a set U belongs to `l₁.map f` if and only if $f^{-1}(U) \in l_1$. The ordering $\leq$ on filters is reverse inclusion of their set of members: $F \leq G$ means that every set in G is also in F. Unwinding the definitions, `Tendsto f l₁ l₂` is equivalent to:

$$\forall\, U \in l_2, \quad f^{-1}(U) \in l_1.$$

In other words, the preimage of every "large" set in l_2 is "large" in l_1.

Example 5.2.1 *We can express convergence of a sequence s_n to a limit a using filters: `Tendsto s atTop (𝓝 a)` means $s_n \to a$ as $n \to \infty$.*

Unwinding the definition: for every neighborhood U of a (i.e., every $U \in \mathcal{N}(a)$), the preimage $s^{-1}(U) = \{n \in \mathbb{N} \mid s_n \in U\}$ is in `atTop`, meaning it contains all sufficiently large n. This recovers the familiar definition: for every open set U containing a, there exists N such that $s_n \in U$ for all $n \geq N$.

5.2.3 From global to local continuity

The definition of `Tendsto` also provides a natural bridge between global continuity (defined in terms of open sets) and local continuity (defined at individual points). As seen before, the structure `Continuous` defines global continuity in terms of preimages of open sets. Mathlib also provides alternative definitions for local continuity: `ContinuousAt` defines continuity at a single point, `ContinuousWithinAt` defines continuity within a set at a point, and `ContinuousOn` defines continuity on an entire set. All these local characterizations are defined in terms of filters.

The connection between global and local continuity is established by the following fundamental theorem:

```
theorem Continuous.tendsto (hf : Continuous f) (x) :
    Tendsto f (𝓝 x) (𝓝 (f x)) :=
  ((nhds_basis_opens x).tendsto_iff <| nhds_basis_opens <| f x).2
    fun t ⟨hxt, ht⟩ => ⟨f ⁻¹' t, ⟨hxt, ht.preimage hf⟩, Subset.rfl⟩
```

The statement `Tendsto f (𝓝 x) (𝓝 (f x))` is a reformulation of continuity at x in terms of neighborhood filters: for every neighborhood U of $f(x)$, the preimage $f^{-1}(U)$ is a neighborhood of x.

The proof uses **filter bases**. Similar to how a topological space can be defined via a basis of open sets, a filter can be defined via a basis of sets. A basis B for a filter F is a nonempty collection of sets that is closed under finite intersections (up to refinement): for any two sets $U, V \in B$, there exists a set $W \in B$ such that $W \subseteq U \cap V$.

```
structure FilterBasis (α : Type*) where
  sets : Set (Set α)
  nonempty : sets.Nonempty
  inter_sets {x y : Set α} : x ∈ sets → y ∈ sets → ∃ z ∈ sets, z ⊆ x ∩
    y
```

Given a basis, we can generate a filter and prove properties about it by working only with basis elements.

To better understand the proof, we can expand it using tactic mode:

```
theorem Continuous'.tendsto (hf : Continuous f) (x) :
    Tendsto f (𝓝 x) (𝓝 (f x)) := by
  rw [(nhds_basis_opens x).tendsto_iff (nhds_basis_opens (f x))]
  intro t ⟨hft_in, ht_open⟩
  use f ⁻¹' t
  constructor
  · exact ⟨hft_in, ht_open.preimage hf⟩
  · exact Subset.rfl
```

The lemma `nhds_basis_opens x` states that the neighborhood filter $\mathcal{N}$ `x` has a basis consisting of all open sets containing x. The rewrite `rw [(nhds_basis_opens x).tendsto_iff (nhds_basis_opens (f x))]` reduces the goal to: for every open neighborhood t of $f(x)$, there exists an open neighborhood s of x with $f(s) \subseteq t$. We introduce an arbitrary open neighborhood t of $f(x)$ using `intro t ⟨hft_in, ht_open⟩`. The natural witness is the preimage `f ⁻¹' t`. The first subgoal, that $x \in f^{-1}(t)$ and $f^{-1}(t)$ is open, follows from `hft_in` and from `ht_open.preimage hf`, which uses the global continuity of f to conclude that the preimage of an open set is open. The second subgoal, that $\forall x \in f^{-1}(t)$, $f(x) \in t$, holds by definition of preimage.

Here is the full example: [link to Lean live]

Using this bridge from global to local continuity, we can understand the following definitions:

```
def ContinuousAt (f : X → Y) (x : X) :=
  Tendsto f (𝓝 x) (𝓝 (f x))

theorem Continuous.continuousAt (h : Continuous f) :
    ContinuousAt f x :=
  h.tendsto x
```

The remaining local continuity concepts are defined similarly:

```
def ContinuousWithinAt (f : X → Y) (s : Set X) (x : X) : Prop :=
  Tendsto f (𝓝[s] x) (𝓝 (f x))

def ContinuousOn (f : X → Y) (s : Set X) : Prop :=
  ∀ x ∈ s, ContinuousWithinAt f s x
```

Note that `𝓝[s] x` denotes the neighborhood filter of x restricted to the set s, allowing us to study the behavior of functions on arbitrary subsets. We will use `ContinuousOn` in the next chapter.

Having introduced the topological machinery available in Mathlib, we now turn to the specific topological concepts needed for our formalization: connectedness and path-connectedness.

5.3 Connectedness and Path-Connectedness

In this section, we recall the definitions of connectedness and path-connectedness from standard topology, following Munkres [Mun00], and present how they are formalized in Mathlib.

Definition 5.3.1 *A topological space X is* ***connected*** *if it cannot be written as the union of two disjoint nonempty open sets. Equivalently, the only subsets of X that are both open and closed are $\varnothing$ and X itself.*

Definition 5.3.2 *A topological space X is* ***path-connected*** *if for every pair of points $a, b \in X$, there exists a continuous map $\gamma : [0, 1] \to X$ (called a* ***path****) such that $\gamma(0) = a$ and $\gamma(1) = b$.*

Figure 5.3.2.1: Visual representation of connectedness vs. path-connectedness.

It is a standard result that path-connectedness implies connectedness [Mun00, Theorem 25.5]. Intuitively, if a path-connected space could be split into two disjoint open sets U and V, then any path from a point in U to a point in V would yield a continuous map from the connected space $[0, 1]$ onto a disconnected space, which is impossible. However, the converse does not hold: a space can be connected without being path-connected. The classical counterexample demonstrating this asymmetry is the **topologist's sine curve**, which we formalize in the next chapter.

5.3.1 Mathlib's formalization

In Mathlib, a subset S of a topological space is connected if it is nonempty and **preconnected**, where preconnected means it cannot be covered by two disjoint open sets that each meet S:

```
def IsPreconnected (s : Set α) : Prop :=
  ∀ (u v : Set α), IsOpen u → IsOpen v → s ⊆ u ∪ v →
    (s ∩ u).Nonempty → (s ∩ v).Nonempty → (s ∩ (u ∩ v)).Nonempty

structure IsConnected (s : Set α) : Prop where
  nonempty : s.Nonempty
  isPreconnected : IsPreconnected s
```

Path-connectedness is defined via the `IsPathConnected` predicate. A subset S is path-connected if there exists a point $x \in S$ such that every other point $y \in S$ can be joined to x by a path lying entirely in S:

```
def IsPathConnected (F : Set X) : Prop :=
  ∃ x ∈ F, ∀ ⦃y⦄, y ∈ F → JoinedIn F x y
```

The auxiliary predicate `JoinedIn` requires the existence of a path between the two points that stays within S:

```
def JoinedIn (S : Set X) (x y : X) : Prop :=
  ∃ γ : Path x y, ∀ t, γ t ∈ S
```

Here, `Path x y` denotes a continuous map $\gamma : [0, 1] \to X$ with $\gamma(0) = x$ and $\gamma(1) = y$. The domain $[0, 1]$ is represented in Lean by the `unitInterval` **subtype** of $\mathbb{R}$, defined as $\{t : \mathbb{R} \mid 0 \leq t \leq 1\}$. As we will see in the next chapter, working with this subtype introduces some friction, since it does not automatically inherit all arithmetic operations from $\mathbb{R}$.

5.3.2 Useful theorems about connectedness

Two results from Mathlib will be especially important for our formalization. The first concerns continuous images:

Theorem 5.3.3 *The continuous image of a connected set is connected.*

In Mathlib, this is available as `IsConnected.image`. As a consequence, `isConnected_Ioi.image` states that the image of an interval (a, ∞) under a continuous map is connected.

The second result concerns closures:

Theorem 5.3.4 *Let C be a connected subset of a topological space. If $C \subseteq S \subseteq \overline{C}$, then S is connected.*

In Mathlib, this is `IsConnected.subset_closure`. Intuitively, adding limit points to a connected set cannot disconnect it, because any open set that meets the added points must also meet the original set.

In this chapter, we have surveyed how Lean and Mathlib formalize topology: from the `TopologicalSpace` type class and the definition of continuity, through filters as a uniform language for convergence, to the definitions of connectedness and path-connectedness. In the next chapter, we put these tools to work by formalizing the topologist's sine curve—a space that is connected but not path-connected.

Chapter 6

Formalizing the Topologist's Sine Curve

In this chapter, we put the tools developed in the previous chapters to work by formalizing a classical counterexample in topology: the **topologist's sine curve**, a space that is connected but not path-connected. We begin by discussing the motivation for choosing this example (Section 6.1), then set up the definitions (Section 6.2). The proof splits into two parts: showing that T is connected (Section 6.3) and showing that T is not path-connected (Section 6.4). We conclude with reflections on the formalization process and directions for future work (Section 6.5).

6.1 Motivation

As part of my thesis work, with the help and revision from Prof. David Loeffler, I formalized the topologist's sine curve in Lean and contributed the result to the Mathlib library. The formalization is available in **Counterexamples – Topologist's Sine Curve**.

The topologist's sine curve is one of the most important counterexamples in point-set topology: it shows that connectedness does not imply path-connectedness (recall Definitions 5.3.1 and 5.3.2). While conceptually simple, the proof involves a mix of topological and analytical techniques, making it a good test of the library's capabilities. Moreover, at the time of this work, Mathlib's `Counterexamples` directory did not yet contain this classic example.

Translating the informal proof into Lean revealed interesting contrasts between pen-and-paper and formal reasoning. Showing that $(0, 0) \in \overline{S}$ using filters closely mirrored the informal argument and went smoothly. The more difficult part was the non-path-connectedness proof, and the difficulty was largely *contingent* on library design rather than *essential* to formal verification. In standard mathematics, we seamlessly apply real arithmetic to the path domain $[0, 1] \subset \mathbb{R}$. In Mathlib, however,

this domain is the `unitInterval` subtype. Because subtypes do not automatically inherit their base type's arithmetic operations, one must constantly cast variables back and forth between `unitInterval` and $\mathbb{R}$, proving at each step that the bounds $[0, 1]$ are satisfied.

A second noteworthy design decision is Mathlib's convention that division by zero yields zero: $1/0 = 0$. While initially alarming to mathematicians, this makes $x \mapsto x^{-1}$ a total function on $\mathbb{R}$, so the identity $(x^{-1})^{-1} = x$ holds unconditionally, including at $x = 0$. This convention simplifies several steps in our proof where nested inverses appear.

6.2 Setup

My original proof follows Conrad's paper ([Con]), with some modifications. The final formalization merged into Mathlib is more general (see the note at the end of Section 6.3).

The topologist's sine curve is defined as the graph of $y = \sin(1/x)$ for $x \in (0, \infty)$, together with the origin $(0, 0)$.

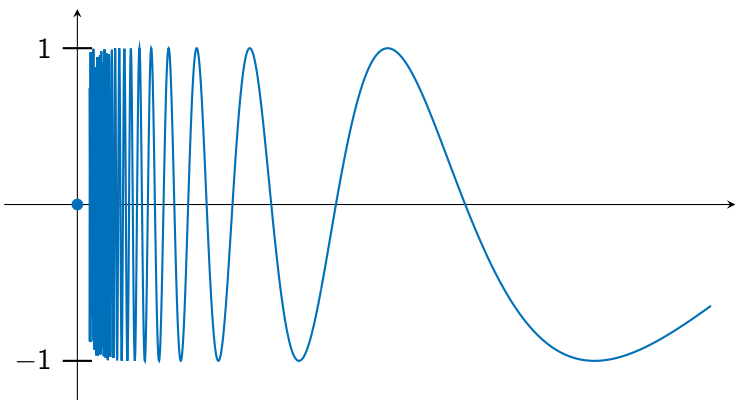

Figure 6.2.0.1: The topologist's sine curve.

We define three sets in $\mathbb{R}^2$:

- S: the oscillating curve $\{(x, \sin(1/x)) : x > 0\}$
- Z: the singleton set $\{(0, 0)\}$
- T: their union $S \cup Z$

In Lean, this is expressed as follows:

```
open Real Set
def pos_real := Ioi (0 : ℝ)
noncomputable def sine_curve := fun x ↦ (x, sin (x⁻¹))
def S : Set (ℝ × ℝ) := sine_curve '' pos_real
def Z : Set (ℝ × ℝ) := { (0, 0) }
def T : Set (ℝ × ℝ) := S ∪ Z
```

We open the `Real` and `Set` namespaces to avoid prefixing operations with `Real.` and `Set.`, respectively. We define the interval $(0, \infty)$ as `pos_real`, using the predefined notation `Ioi 0` from `Set`. The function `sine_curve` maps a real number to the corresponding point on the curve in $\mathbb{R}^2$. The notation `''` denotes the image of a set under a function. It is marked `noncomputable` because it involves the sine function, which relies on classical axioms as discussed in Section 5.1. The sets are subsets of the product space $\mathbb{R}^2$, represented as $\mathbb{R} \times \mathbb{R}$.

The goal is to prove that T is connected but not path-connected.

6.3 T is Connected

The informal argument proceeds as follows. The set S is connected, since it is the image of the connected set $(0, \infty)$ under the continuous map $x \mapsto (x, \sin(1/x))$ (Theorem 5.3.3). Moreover, the closure of a connected set is connected, and every set lying between a connected set and its closure is connected (Theorem 5.3.4). Since T is contained in the closure of S, it follows that T is connected.

In a formal proof, each step must be justified. For instance, justifying that S is connected requires proving that the map $x \mapsto (x, \sin(1/x))$ is continuous on $(0, \infty)$ and that $(0, \infty)$ is connected. As we saw even for rational numbers, seemingly simple facts can require several steps and lemmas from Mathlib. Let us walk through the proof.

We use the `IsConnected` structure, introduced in Section 5.3, to set up the statement:

```
lemma S_is_conn : IsConnected S := by sorry
```

In the Mathlib file where `IsConnected` is defined (`Topology/Connected/Basic.lean`), we see that it requires S to be nonempty and preconnected. One can verify this by unfolding `IsConnected` in the goal:

```
lemma S_is_conn : IsConnected S := by
  unfold IsConnected
  -- ⊢ S.Nonempty ∧ IsPreconnected S
  sorry
```

The `unfold` tactic expands definitions, helping us see what Lean expects. Reflecting our informal argument, we check whether Mathlib contains the fact that every interval is connected and that connectedness is preserved under continuous maps. Indeed, in `Topology/Connected/Interval.lean`, we find the theorem `isConnected_Ioi.image`, stating that the image of an interval of the form (a, ∞) under a continuous map is connected:

```
lemma S_is_conn : IsConnected S := by
  apply isConnected_Ioi.image
  -- ⊢ ContinuousOn sine_curve (Ioi 0)
  sorry
```

It remains to prove `ContinuousOn sine_curve (Ioi 0)`: the function is continuous on $(0, \infty)$. The function $x \mapsto (x, \sin(1/x))$ is continuous on $(0, \infty)$ as the product of two functions continuous on this domain: the identity map $x \mapsto x$ and the composition $x \mapsto \sin(1/x)$. Here is the full proof:

```
lemma inv_is_continuous_on_pos_real :
    ContinuousOn (fun x : ℝ => x⁻¹) pos_real := by
  apply ContinuousOn.inv₀
  · exact continuous_id.continuousOn
  · intro x hx; exact ne_of_gt hx

lemma sin_comp_inv_is_continuous_on_pos_real :
    ContinuousOn sine_curve pos_real := by
  apply ContinuousOn.prodMk continuous_id.continuousOn
  apply continuous_sin.comp_continuousOn
  exact inv_is_continuous_on_pos_real
```

Starting from the bottom lemma, `ContinuousOn.prodMk` states that the product of two functions continuous on a set is continuous on that set, requiring a proof of the continuity of each component. The first component is the identity map, which is continuous on any set; Mathlib provides `continuous_id.continuousOn` for this. The second component is the composition of sine with the inverse. The theorem `Continuous.comp_continuousOn` states that the composition of a globally continuous function with a function continuous on a set is continuous on that set. We separate the proof that the inverse function is continuous on $(0, \infty)$ into the auxiliary lemma `inv_is_continuous_on_pos_real`. The theorem `ContinuousOn.inv₀` requires that the function is continuous and nonzero on the set. The nonzero condition is discharged

by:

```
  · intro x hx
    exact ne_of_gt hx
```

The hypothesis `hx` states that $x \in (0, \infty)$, i.e., $x > 0$. The lemma `ne_of_gt` states that a positive number is nonzero. Thus the final proof is:

```
lemma S_is_conn : IsConnected S := by
  apply isConnected_Ioi.image
  · exact sin_comp_inv_is_continuous_on_pos_real
```

6.3.1 Refactoring: from tactics to terms

When writing a proof, one typically starts by working out the informal argument on paper, then translates it into Lean step by step, searching for theorems in Mathlib. Afterwards, one can refactor the proof to remove unnecessary steps. Let us showcase this process. The auxiliary lemmas can be reduced to term-mode proofs:

```
lemma inv_is_continuous_on_pos_real :
    ContinuousOn (fun x : ℝ => x⁻¹) pos_real :=
  ContinuousOn.inv₀ continuous_id.continuousOn
    (fun _ hx => ne_of_gt hx)

lemma sin_comp_inv_is_continuous_on_pos_real :
    ContinuousOn sine_curve pos_real :=
  ContinuousOn.prodMk continuous_id.continuousOn <|
    continuous_sin.comp_continuousOn <|
      inv_is_continuous_on_pos_real
```

We removed the `by` keyword since we can provide a **term** that directly proves the statement (recall the propositions-as-types correspondence from Chapter **??**). In the second lemma, we use the reverse application operator `<|`, which allows us to avoid parentheses: `f <| g <| h` is equivalent to `f (g h)`.

We can inline both lemmas to obtain a single proof term:

```
lemma S_is_conn : IsConnected S :=
  isConnected_Ioi.image sine_curve <|
    continuous_id.continuousOn.prodMk <|
    continuous_sin.comp_continuousOn <|
    ContinuousOn.inv₀ continuous_id.continuousOn
      (fun _ hx => ne_of_gt hx)
```

Reading from left to right (thanks to the `<|` operator), the proof builds up by successive applications:

- `isConnected_Ioi.image sine_curve` states that the image of $(0, \infty)$ under `sine_curve` is connected, provided the function is continuous on that domain.
- `continuous_id.continuousOn.prodMk` constructs the continuity of a product from the continuity of each component.
- `continuous_sin.comp_continuousOn` provides the continuity of the sine composition.
- `ContinuousOn.inv₀` ... proves the continuity of the inverse on positive reals.

The chain reads as building the continuity proof from the innermost function (the inverse) outward to the complete sine curve function.

Providing explicit proof terms rather than using tactics is not merely an aesthetic choice. Lean compiles such proofs faster because it does not need to run the tactic interpreter. This practice is common in Mathlib, where the community balances readability with compilation performance.

6.3.2 From S to T

Since $Z \cap S = \varnothing$, we cannot directly conclude that T is connected from the connectedness of its components. However, by Theorem 5.3.4, every subset between a connected set and its closure is connected. In Mathlib, this theorem is available as `IsConnected.subset_closure`:

```
theorem T_is_conn : IsConnected T := by
  apply IsConnected.subset_closure
  · exact S_is_conn        -- ⊢ IsConnected S
  · tauto_set               -- ⊢ S ⊆ T
  · sorry                   -- ⊢ T ⊆ closure S
```

The theorem requires three goals: that S is connected (already proved), that $S \subseteq T$ (a trivial set operation handled by `tauto_set`), and that $T \subseteq \overline{S}$ (the closure of S), which requires proof.

```
lemma T_sub_cls_S : T ⊆ closure S := by
  intro x hx
  cases hx with
  | inl hxS => exact subset_closure hxS
  | inr hxZ => sorry
```

To show set containment, we introduce an element $x \in T$ and consider cases. When $x \in S$, the result follows immediately from `subset_closure`. The case $x \in Z$ requires more work: we must show that the origin lies in the closure of S.

At this point in the formalization, one needs to find relevant Mathlib lemmas. A useful tool is Loogle (similar to Haskell's Hoogle), which searches Mathlib theorems by type signature or name patterns. Separately, Lean 4 has powerful built-in search capabilities. By appending a question mark to certain tactics, Lean automatically searches the imported library for applicable lemmas. For instance, `exact?` (or `apply?`) searches for theorems that can directly close (or make progress on) the current goal. The tactics `rw?` and `simp?` work similarly for rewriting and simplification.

At this point, `apply?` suggests several ways to proceed. Some use filters:

```
Try this: refine Frequently.mem_closure ?_
Remaining subgoals:
  ⊢ ∃ᶠ (x : ℝ × ℝ) in 𝓝 x, x ∈ S
```

Others follow the more familiar metric space approach:

```
Try this: refine Metric.mem_closure_iff.mpr ?_
Remaining subgoals:
  ⊢ ∀ ε > 0, ∃ b ∈ S, dist x b < ε
```

The best approach, however, is to think about the mathematics first. Since we are working with a metrizable topology on $\mathbb{R}^2$, the closure of a set contains all its limit points. To show that $(0,0) \in \overline{S}$, it suffices to construct a sequence in S converging to $(0,0)$. We use the filter-based approach from Section 5.2:

```
lemma T_sub_cls_S : T ⊆ closure S := by
  intro x hx
  cases hx with
  | inl hxS => exact subset_closure hxS
  | inr hxZ =>
      rw [hxZ]
      -- Define sequence: f(n) = (1/(nπ), 0)
      let f : ℕ → ℝ × ℝ := fun n => ((n * Real.pi)⁻¹, 0)
      -- Show f converges to (0, 0)
      have hf : Tendsto f atTop (𝓝 (0, 0)) := by
        refine Tendsto.prodMk_nhds ?_ tendsto_const_nhds
        exact tendsto_inv_atTop_zero.comp
          (Tendsto.atTop_mul_const' Real.pi_pos
            tendsto_natCast_atTop_atTop)
      -- Show f eventually takes values in S
      have hf' : ∀ᶠ n in atTop, f n ∈ S := by
        filter_upwards [eventually_gt_atTop 0] with n hn
        exact ⟨(n * Real.pi)⁻¹,
          inv_pos.mpr (mul_pos (Nat.cast_pos.mpr hn) Real.pi_pos),
          by simp [f, sine_curve, inv_inv, Real.sin_nat_mul_pi]⟩
      -- Apply sequential characterization of closure
      exact mem_closure_of_tendsto hf hf'
```

Let us break down the key steps. Using `let`, we define $f(n) = \left(\frac{1}{n\pi}, 0\right)$, which converges to $(0, 0)$ and eventually lies in S.

- **Convergence** (`hf`): We show `Tendsto f atTop (𝓝 (0, 0))` (recall from Section 5.2 that this means $f(n) \to (0, 0)$ as $n \to \infty$). Using `Tendsto.prodMk_nhds`, we reduce this to showing convergence of each coordinate separately. For the first coordinate, we compose `tendsto_inv_atTop_zero` (which states $1/x \to 0$ as $x \to \infty$) with the fact that $n\pi \to \infty$. The second coordinate is constant, handled by `tendsto_const_nhds`.

- **Membership** (`hf'`): We show `∀ᶠ n in atTop, f n ∈ S`, meaning $f(n) \in S$ for all sufficiently large n. We use `filter_upwards`, which allows combining hypotheses about properties that hold eventually, together with `eventually_gt_atTop 0` (which states that eventually $n > 0$). For such n, we show $f(n) = (1/(n\pi), 0) \in S$ by noting:

$$\sin\left(\frac{1}{1/(n\pi)}\right) = \sin(n\pi) = 0.$$

  Here the convention $1/0 = 0$ is helpful: the identity `inv_inv` (stating $(x^{-1})^{-1} =$

x) holds unconditionally, so `simp` resolves the nested inverses without requiring a side-proof of nonzero-ness.

Finally, `mem_closure_of_tendsto` combines these facts: if a sequence eventually stays in S and converges to x, then $x \in \overline{S}$.

Assembling the connectedness proof

We can now assemble the full proof. In term mode:

```
theorem T_is_conn : IsConnected T :=
  IsConnected.subset_closure
    (isConnected_Ioi.image sine_curve <|
      continuous_id.continuousOn.prodMk <|
      continuous_sin.comp_continuousOn <|
      ContinuousOn.inv₀ continuous_id.continuousOn
        (fun _ hx => ne_of_gt hx))
    (by tauto_set) T_sub_cls_S
```

Here is the link to the entire first part of the proof: [link to Lean live]

Note 6.3.1 *The proof merged into Mathlib takes Z as $\{0\} \times [-1, 1]$ instead of the singleton $\{(0, 0)\}$. Together with the fact that T equals the closure of S, this yields a stronger result: the closure of S is connected but not path-connected, showing that forming the closure can destroy path-connectedness.*

6.4 T is not Path-Connected

The main and most substantial part of the formalization is showing that T is not path-connected. Even the informal argument requires careful construction. We proceed by contradiction. Suppose a continuous path $p : [0, 1] \to T$ connects a point in Z to a point in S. As the path approaches the y-axis (where $x \to 0$), the y-coordinate must oscillate infinitely between -1 and 1 due to the behavior of $\sin(1/x)$ as $x \to 0^+$. This infinite oscillation contradicts the continuity of the path.

Recall from Section 5.3 that `IsPathConnected` requires a witness path between any two points in the set, using the `JoinedIn` predicate and the `Path` structure.

We begin the proof by assuming path-connectedness and extracting a path:

```
theorem T_is_not_path_conn : ¬ IsPathConnected T := by
  -- Fix two points: z = (0, 0) and w = (1, sin(1))
  have hz : z ∈ T := Or.inr rfl
  have hw : w ∈ T := Or.inl ⟨1, ⟨zero_lt_one' ℝ, rfl⟩⟩
  intro p_conn
  apply IsPathConnected.joinedIn at p_conn
  specialize p_conn z hz w hw
  let p := JoinedIn.somePath p_conn
```

We fix two points: $z = (0, 0) \in Z$ and $w = (1, \sin(1)) \in S$, and prove they are both in T. Using `intro p_conn`, we assume that T is path-connected; the goal becomes `False`. The last three lines extract an explicit path `p` connecting z and w: `apply IsPathConnected.joinedIn at p_conn` transforms the assumption into the statement that any two points in T are joined; `specialize p_conn z hz w hw` specializes this to z and w; and `let p := JoinedIn.somePath p_conn` extracts the path.

6.4.1 Defining t_0 and the continuity argument

Conrad's paper ([Con]) defines $t_0 = \inf\{t \in [0, 1] : x(p(t)) > 0\}$, the first time the x-coordinate of p becomes positive. To simplify some technical steps, we instead define

$$t_0 = \sup\{t \in [0, 1] : x(p(t)) = 0\},$$

the *last* time the path remains on the y-axis. By continuity at t_0, we can find a neighborhood where the path stays close to $p(t_0) = (0, 0)$. Specifically, with $\varepsilon = 1/2$, there exists $\delta > 0$ such that for all t with $|t - t_0| < \delta$, $\|p(t) - p(t_0)\| < 1/2$.

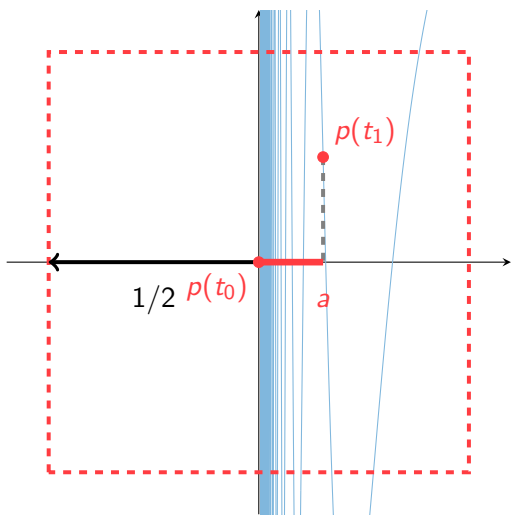

Figure 6.4.0.1: The ε-ball around $p(t_0)$. The ball is depicted as a square because Mathlib uses the L^∞ norm on $\mathbb{R}^2$.

```
-- The x-coordinate composed with p is continuous
have xcoord_pathcont : Continuous fun t ↦ (p t).1 :=
  continuous_fst.comp p.continuous
-- t₀ = last time the path is on the y-axis
let t₀ : unitInterval := sSup {t | (p t).1 = 0}
let xcoord_path := fun t => (p t).1
-- The x-coordinate at t₀ is 0
have hpt₀_x : (p t₀).1 = 0 :=
  (isClosed_singleton.preimage xcoord_pathcont).sSup_mem
    ⟨0, by aesop⟩
-- By continuity: ∃ δ > 0, ∀ t, dist t t₀ < δ →
--   dist (p t) (p t₀) < 1/2
obtain ⟨δ, hδ, ht⟩ : ∃ δ > 0, ∀ t, dist t t₀ < δ →
    dist (p t) (p t₀) < 1/2 :=
  Metric.eventually_nhds_iff.mp <|
    Metric.tendsto_nhds.mp (p.continuousAt t₀) _ one_half_pos
```

The `obtain` tactic destructures the existential quantifier into δ, the proof `hδ` that $\delta > 0$, and `ht` (the distance condition). We use the metric characterization of continuity (Section 5.2): `p.continuousAt t₀` gives continuity at t_0; `Metric.tendsto_nhds.mp` converts this to the ε-δ form; and `Metric.eventually_nhds_iff.mp` unpacks it into $\forall\, t,\ \mathrm{dist}(t, t_0) < \delta \to \mathrm{dist}(p(t), p(t_0)) < 1/2$.

6.4.2 Finding t_1 and the positive x-coordinate

We next find a time $t_1 > t_0$ within δ of t_0 where the x-coordinate becomes positive, and set $a = x(p(t_1)) > 0$:

```
-- Find t₁ > t₀ with dist t₀ t₁ < δ
obtain ⟨t₁, ht₁⟩ : ∃ t₁, t₁ > t₀ ∧ dist t₀ t₁ < δ := by
  let s₀ := (t₀ : ℝ)
  let s₁ := min (s₀ + δ/2) 1
  have hs₁ : 0 ≤ s₁ := le_min (add_nonneg t₀.2.1 (by positivity))
    zero_le_one
  have hs₁': s₁ ≤ 1 := min_le_right ..
  sorry
-- a = x(p(t₁)) > 0
let a := (p t₁).1
have ha : a > 0 := by
  obtain ⟨x, hxI, hx_eq⟩ : p t₁ ∈ S := by
    cases p_conn.somePath_mem t₁ with
    | inl hS => exact hS
    | inr hZ =>
      have : (p t₁).1 = 0 := by rw [hZ]
      have hle : t₁ ≤ t₀ := le_sSup this
      have : (t₁ : ℝ) ≤ (t₀ : ℝ) := Subtype.coe_le_coe.mpr hle
      have : (t₁ : ℝ) > (t₀ : ℝ) := Subtype.coe_lt_coe.mpr ht₁.1
      linarith
  simpa only [a, ← hx_eq] using hxI
```

This code is the most technically involved part of the proof, and the difficulty is largely due to working with the `unitInterval` subtype. To construct t_1, we cast t_0 to $\mathbb{R}$ (`let s₀ := (t₀ : ℝ)`), compute $s_1 = \min(s_0 + \delta/2, 1)$, and then cast back to `unitInterval` by providing proofs that $0 \leq s_1$ and $s_1 \leq 1$. This back-and-forth casting is a recurring source of friction, as noted in Section 6.1.

To show $a > 0$, we argue that $p(t_1) \in S$ (and hence has positive x-coordinate). Since p stays in $T = S \cup Z$, there are two cases. If $p(t_1) \in S$, we are done. If $p(t_1) \in Z$, then $(p(t_1))_1 = 0$, which would imply $t_1 \leq t_0$ by the definition of t_0 as the supremum. But $t_1 > t_0$ by construction—a contradiction, dispatched by `linarith` after transferring the inequalities from `unitInterval` to $\mathbb{R}$ via `Subtype.coe_le_coe` and `Subtype.coe_lt_coe`.

6.4.3 The interval argument

Next, we observe that the image $x(p([t_0, t_1]))$ is connected (as the continuous image of a connected set) and contains both $0 = x(p(t_0))$ and $a = x(p(t_1))$. Since every connected subset of $\mathbb{R}$ is an interval (a consequence of the intermediate value theorem), we obtain:

$$[0, a] \subseteq x\big(p([t_0, t_1])\big).$$

```
have Icc_sub : Set.Icc 0 a ⊆ xcoord_path '' Set.Icc t₀ t₁ :=
  IsConnected.Icc_subset
    ((isConnected_Icc (le_of_lt ht₁.1)).image _
      xcoord_pathcont.continuousOn)
    (⟨t₀, left_mem_Icc.mpr (le_of_lt ht₁.1), hpt₀_x⟩)
    (⟨t₁, right_mem_Icc.mpr (le_of_lt ht₁.1), rfl⟩)
```

The lemma `IsConnected.Icc_subset` states that if a connected subset of $\mathbb{R}$ contains two values c and d, then it contains the entire interval $[c, d]$. We apply it to the image of $[t_0, t_1]$ under the x-coordinate map, which is connected (as the continuous image of an interval) and contains 0 and a.

6.4.4 Constructing the oscillating sequence

Now we construct a sequence that demonstrates the contradiction. Recall that $\sin(\theta) = 1$ if and only if $\theta = (4k + 1)\pi/2$ for some integer k. Therefore, $\sin(1/x) = 1$ when

$$x = \frac{2}{(4k + 1)\pi}$$

for $k \in \mathbb{N}$. As $k \to \infty$, these x-values approach 0, so infinitely many of them lie in any interval $[0, a]$.

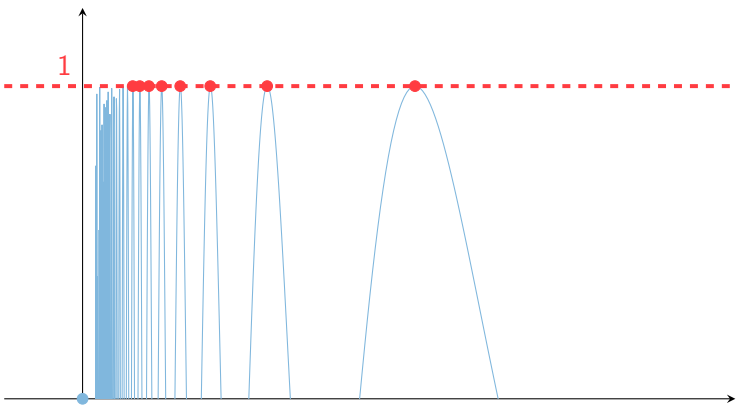

Figure 6.4.0.2: The sequence `xs_pos_peak` isolates the points where the curve reaches $y = 1$, accumulating toward the origin.

We define this sequence and establish its key properties:

```
noncomputable def xs_pos_peak :=
  fun (k : ℕ) => 2 / ((4 * k + 1) * Real.pi)

lemma xs_pos_peak_tendsto_zero :
    Tendsto xs_pos_peak atTop (𝓝 0) := sorry

lemma xs_pos_peak_nonneg :
    ∀ k : ℕ, 0 ≤ xs_pos_peak k := sorry

lemma sin_xs_pos_peak_eq_one (k : ℕ) :
    Real.sin ((xs_pos_peak k)⁻¹) = 1 := sorry
```

The proofs of these auxiliary lemmas (omitted for brevity) are routine calculations. The crucial consequence is that this sequence eventually enters $[0, a]$:

```
have xpos_in_Icc : ∃ i : ℕ, i ≥ 1 ∧
    xs_pos_peak i ∈ Set.Icc 0 a := sorry
```

This follows from `xs_pos_peak_tendsto_zero`: since the sequence converges to 0 and $a > 0$, the terms eventually lie in $[0, a]$.

6.4.5 The final contradiction

We can now assemble the contradiction. Since $[0, a] \subseteq x(p([t_0, t_1]))$ and $\mathrm{xs_pos_peak}(i) \in [0, a]$ for some i, there must exist $t' \in [t_0, t_1]$ with $x(p(t')) = \mathrm{xs_pos_peak}(i)$. Because $p(t') \in T$ and $x(p(t')) > 0$, the point $p(t')$ must lie in S (not in Z), so $p(t') = (\mathrm{xs_pos_peak}(i), \sin(1/\mathrm{xs_pos_peak}(i))) = (\mathrm{xs_pos_peak}(i), 1)$. Hence the y-coordinate of $p(t')$ equals 1.

However, since $t' \in [t_0, t_1]$ and $\mathrm{dist}(t_0, t_1) < \delta$, we have $\mathrm{dist}(t', t_0) < \delta$, and by our continuity argument:

$$\|p(t') - p(t_0)\| < \tfrac{1}{2}.$$

But $p(t_0) = (0, 0)$ and $(p(t'))_2 = 1$, so:

$$\|p(t') - p(t_0)\| \geq |(p(t'))_2 - (p(t_0))_2| = |1 - 0| = 1 > \tfrac{1}{2}.$$

This is a contradiction.

```
-- Extract t' ∈ [t₀, t₁] with (p t').2 = 1
obtain ⟨t', ht', hpath_t'⟩ :
    ∃ t' ∈ Set.Icc t₀ t₁, (p t').2 = 1 := sorry
-- p(t₀) = (0, 0)
have hpt₀ : p t₀ = (0, 0) := sorry
-- t' is within δ of t₀
have t'_close : dist t' t₀ < δ := by
  calc dist t' t₀
      ≤ dist t₁ t₀ :=
        dist_right_le_of_mem_uIcc (Icc_subset_uIcc' ht')
    _ = dist t₀ t₁ := dist_comm _ _
    _ < δ := ht₁.2
-- Upper bound from continuity
have close : dist (p t') (p t₀) < 1/2 := ht t' t'_close
-- Lower bound from geometry
have far : 1 ≤ dist (p t') (p t₀) := by
  calc 1
      = |(p t').2 - (p t₀).2| := by simp [hpath_t', hpt₀]
    _ ≤ ‖p t' - p t₀‖ := norm_ge_abs_snd
    _ = dist (p t') (p t₀) := by rw [dist_eq_norm]
-- Contradiction: 1 ≤ dist … < 1/2
linarith
```

The `calc` chain for `t'_close` shows that $\mathrm{dist}(t', t_0) \leq \mathrm{dist}(t_1, t_0)$ (since $t' \in [t_0, t_1]$), which equals $\mathrm{dist}(t_0, t_1) < \delta$ by construction. The chain for `far` uses `norm_ge_abs_snd` (the norm of a pair dominates the absolute value of its second component) and `dist_eq_norm` (distance equals norm of the difference). Finally, `linarith` observes the contradiction $1 \leq \mathrm{dist}(p(t'), p(t_0)) < 1/2$.

6.5 Conclusion

We combine the two parts in the following theorem:

```
theorem T_is_conn_not_pathconn :
    IsConnected T ∧ ¬IsPathConnected T :=
  ⟨T_is_conn, T_is_not_path_conn⟩
```

Since this code compiles successfully, these two lines stand as verified witnesses to the correctness of our entire proof. This showcases the power of proof assistants: mathematics becomes not only more rigorous but also automatically verifiable. The formalization also serves as a learning tool—future readers can inspect each step of the code and verify it interactively.

Here is the full proof: [link to Lean live]

6.5.1 Reflections on the formalization

Looking back at the formalization, several observations stand out.

The connectedness proof translated naturally into Lean. Mathlib's library of results about connected sets, continuous images, and closures meant that the formal argument closely mirrored the informal one. The filter-based proof that $(0, 0) \in \overline{S}$ was particularly clean: the Mathlib lemma `mem_closure_of_tendsto` encapsulates exactly the mathematical argument (a converging sequence that eventually stays in the set).

The non-path-connectedness proof was substantially more involved. The most time-consuming aspect was managing the `unitInterval` subtype. In pen-and-paper mathematics, we freely perform arithmetic on elements of $[0, 1]$ and appeal to their properties as real numbers. In Lean, every such operation requires casting to $\mathbb{R}$, performing the computation, and casting back with explicit proofs that the result lies in $[0, 1]$. This accounts for a substantial fraction of the code but reflects a design choice in Mathlib rather than a fundamental limitation of formal verification. Future improvements to the subtype API or automation for bound checking could significantly reduce this overhead.

As noted in Section 6.1, Mathlib's convention that $1/0 = 0$ proved helpful in several places. The identity $(x^{-1})^{-1} = x$ (`inv_inv`) holds unconditionally, which allowed `simp` to resolve nested inverses without side-proofs of nonzero-ness. This illustrates how design decisions in a formal library can have unexpected downstream benefits.

6.5.2 Future directions

Formalizing counterexamples like the topologist's sine curve paves the way for deeper developments in algebraic topology within Lean.

The Warsaw circle. A natural next step is formalizing the **Warsaw circle**; a closed variant obtained by connecting the endpoints of the topologist's sine curve with an arc. This space famously has a trivial fundamental group $\pi_1(X, x_0) = 0$ but a nontrivial first singular homology group $H_1(X) \neq 0$, demonstrating that these two invariants can disagree for spaces that are not well-behaved.

Homotopy theory. Path-connectedness is intimately tied to homotopies: two paths are **homotopic** if one can be continuously deformed into the other. The topologist's sine curve provides a natural testing ground for Mathlib's developing homotopy API. In particular, one could formalize the fact that the fundamental group of T is trivial (since T is simply connected despite not being path-connected in

the traditional sense. More precisely, any loop in T must stay on S and is therefore contractible).

In this chapter, we have formalized a complete proof that the topologist's sine curve is connected but not path-connected. The formalization illustrates both the power and the practical challenges of working with a proof assistant: while the mathematical ideas translate naturally into Lean, managing subtypes and type coercions requires significant effort. The result has been merged into Mathlib's `Counterexamples` directory, contributing a modest but concrete example to the growing body of formalized mathematics.

Bibliography

[Alg19] Algebrology. *Constructing the Rational Numbers, Part 1*. Blog post. 2019. URL: `https://algebrology.github.io/constructing-the-rational-numbers-1/` (visited on 01/15/2024).

[Car19] Mario Carneiro. "The Type Theory of Lean". MA thesis. Carnegie Mellon University, 2019. URL: `https://github.com/digama0/lean-type-theory/releases`.

[Con] Keith Conrad. *Spaces that are connected but not path-connected*. `https://kconrad.math.uconn.edu/blurbs/topology/connnotpathconn.pdf`.

[Lea20] Lean Mathematical Library community. "The Lean Mathematical Library". In: 2020.

[Lea25] Lean Mathematical Library community. *Mathlib Documentation*. `https://leanprover-community.github.io/mathlib4_docs`. 2025.

[LM20] Robert Y. Lewis and Paul-Nicolas Madelaine. "Simplifying Casts and Coercions". In: *arXiv preprint arXiv:2001.10594* (2020), pp. 3–4.

[Mun00] James R. Munkres. *Topology*. 2nd ed. Prentice Hall, 2000.

[NPS90] Bengt Nordstr"om, Kent Petersson, and Jan M. Smith. *Programming in Martin-L"of's Type Theory: An Introduction*. Oxford University Press, 1990.

[Tho99] Simon Thompson. *Type Theory And Functional Programming*. University of Kent, 1999.

[Wad15] Philip Wadler. "Propositions as Types". In: *Communications of the ACM* 58.12 (2015), pp. 75–84.

[WB89] Philip Wadler and Stephen Blott. "How to Make Ad-Hoc Polymorphism Less Ad Hoc". In: *16th ACM Symposium on Principles of Programming Languages*. 1989.